

华中科技大学

课程实验报告

课程名称： 数据结构实验

专业班级 CS 2409

学 号 U202490042

姓 名 马俊豪

指导教师 李剑军

报告日期 2025 年 6 月 13 日

计算机科学与技术学院

华中科技大学课程实验报告

目 录

实验 1 基于顺序存储结构的线性表实现	1
1.1 问题描述	1
1.2 系统设计	1
1.3 系统实现	3
1.4 系统测试	13
1.5 实验小结	15
实验 2 基于邻接表的图实现	16
2.1 问题描述	16
2.2 系统设计	16
2.3 系统实现	18
2.4 系统测试	28
2.5 实验小结	31
3 课程的收获和建议	32
3.1 基于顺序存储结构的线性表实现	32
3.2 基于链式存储结构的线性表实现	32
3.3 基于二叉链表的二叉树实现	33
3.4 基于邻接表的图实现	33
参考文献	35
附录 A 基于顺序存储结构线性表实现的源程序	36
附录 B 基于链式存储结构线性表实现的源程序	54
附录 C 基于二叉链表二叉树实现的源程序	75
附录 D 基于邻接表图实现的源程序	101

实验 1 基于顺序存储结构的线性表实现

1.1 问题描述

线性表是一种重要的数据结构，由一组有序排列的元素组成，每个元素都有一个独特的索引值。在本实验中，我们将探讨基于顺序存储结构的线性表的基本概念和操作，包括插入、删除和查找等。

为了更好地理解线性表的工作原理，本实验将开发一个简单的功能演示系统。这个系统包含多个菜单选项，用户可以通过选择不同的选项来执行各种操作，从而增强对线性表的掌握和应用能力

1.2 系统设计

系统通过一个大的 `while` 循环实现菜单选择功能。每次进入循环时，使用变量 `op` 来读取用户的选择，根据 `op` 的值使用 `switch` 函数跳转到相应的功能函数并执行对应操作。当 `op` 为 0 时，退出循环。

进入循环后，系统会打印数字 0-20 对应的功能菜单，并提示用户输入 0-20 范围内的数字。以下是各选项对应的功能函数：

1. 初始化表 (`InitList`)
2. 销毁表 (`DestroyList`)
3. 清空表 (`ClearList`)
4. 判定空表 (`ListEmpty`)
5. 求表长 (`ListLength`)
6. 获得元素 (`GetElem`)
7. 查找元素 (`LocateElem`)
8. 获得前驱 (`PriorElem`)
9. 获得后继 (`NextElem`)
10. 插入元素 (`ListInsert`)
11. 删除元素 (`ListDelete`)
12. 遍历表 (`ListTraverse`)
13. 最大连续子数组和 (`Maximum Subarray`)
14. 和为 K 的子数组 (`Subarray Sum Equals K`)
15. 顺序表排序 (`Sort List`)
16. 保存线性表到文件 (`Save List to File`)
17. 建立新表 (`Create New List`)
18. 删除表 (`Delete List`)

19. 切换表 (Switch List)

通过上述菜单选项，用户可以选择不同的功能进行操作，系统将根据用户的选择执行相应的处理逻辑，从而实现对线性表的各种操作。

在设计线性表时，需要解决以下具体问题：

- 存储结构的选择：选择顺序存储结构还是链式存储结构，需权衡存储效率和操作效率。
- 容量的动态扩展：当线性表存储空间不足时，如何动态扩展存储容量以容纳更多元素。
- 基本操作的实现：包括插入、删除、查找、更新等操作的具体实现及其时间复杂度优化。
- 边界条件处理：如何处理空表、满表以及非法操作（如越界访问）等特殊情况。
- 数据类型的通用性：设计线性表时，如何支持存储多种数据类型（如整数、浮点数、字符串等）。
- 内存管理：如何高效管理内存，避免内存泄漏或冗余分配。
- 算法效率：针对不同操作需求，优化算法以提高线性表的整体性能。

1.2.1 一些常量定义

```
#include "stdio.h"
#include "stdlib.h"
#define TRUE 1
#define FALSE 0
#define OK 1
#define ERROR 0
#define INFEASIBLE -1
#define OVERFLOW -2

typedef int status;
typedef int ElemType; // 数据元素类型定义

#define LIST_INIT_SIZE 100
#define LISTINCREMENT 10
typedef int ElemType;
typedef struct
{ // 顺序表（顺序结构）的定义
    ElemType *elem;
    int length;
    int listsize;
```

```
} SqlList;
typedef struct
{ // 线性表的管理表定义
    struct
    {
        char name[30];
        SqlList L;
    } elem[10];
    int length;
    int listsize;
} LISTS;
```

结构体包含三个元素，每个元素的作用如下：

elem: 这是一个指针，指向顺序表的首元素。通过这个指针，我们可以访问和操作顺序表中的各个元素。

length: 这是一个整数，表示顺序表当前包含的元素数量。它反映了顺序表的实际长度。 **listsize:** 这是一个整数，表示顺序表当前所分配的内存大小。它用于管理内存的动态分配，确保顺序表在需要时可以扩展。

通过定义 `SqlList` 结构体，我们成功实现了顺序表从逻辑结构到物理结构的映射。这为我们后续实现各种操作提供了基础。接下来，我将详细阐述线性表所具有的各种运算功能，并说明它们的实现思路。

1.3 系统实现

1.3.1 初始化表

函数名称：‘`InitList(L)`’

初始条件：线性表 `L` 不存在

操作结果：构建一个空的线性表操作步骤：

1. 检查 `L.elem` 是否为 `NULL`，若不是，则返回 `ERROR`。
2. 为顺序表分配大小为 `LIST_INIT_SIZE` 的初始空间，将 `listsize` 设置为 `LIST_INIT_SIZE`，并将 `length` 设置为 0。

1.3.2 销毁表

函数名称：‘`DestroyList(L)`’初始条件：线性表 `L` 已存在操作结果：销毁线性表 `L` 操作步骤：

1. 检查 ‘`L.elem`’ 是否为 ‘`NULL`’，若是，则返回 ‘`ERROR`’。
2. 使用 ‘`free(L.elem)`’ 释放顺序表的内存，并将 ‘`L.length`’ 和 ‘`L.listsize`’ 置

为 0。

1.3.3 清空表

函数名称：‘ClearList(L)’初始条件：线性表 L 已存在操作结果：将 L 重置为空表操作步骤：

1. 检查 ‘L.elem’ 是否为 ‘NULL’，若不是，则返回 ‘ERROR’。
2. 将 ‘L.listsize’ 设置为 0。

1.3.4 判定空表

函数名称：‘ListEmpty(L)’初始条件：线性表 L 已存在
操作结果：如果 L 为空表则返回 ‘TRUE’，否则返回 ‘FALSE’操作步骤：

1. 检查 ‘L.elem’ 是否为 ‘NULL’，若不是，则返回 ‘ERROR’。
2. 若 ‘L.length’ 为 0，则返回 ‘TRUE’，否则返回 ‘FALSE’。

1.3.5 求表长

函数名称：‘ListLength(L)’
初始条件：线性表已存在
操作结果：返回 L 中数据元素的数量操作步骤：

1. 检查 ‘L.elem’ 是否为 ‘NULL’，若不是，则返回 ‘ERROR’。
2. 返回 ‘L.length’。

1.3.6 获得元素

函数名称：‘GetElem(L, i, e)’
初始条件：线性表已存在， $1 \leq i \leq \text{ListLength}(L)$
操作结果：用 ‘e’ 返回 L 中第 ‘i’ 个数据元素的值操作步骤：

1. 检查 ‘L.elem’ 是否为 ‘NULL’，若不是，则返回 ‘ERROR’。
2. 判断 ‘i’ 是否在区间 [1, L.length] 内，若不在，则返回 ‘ERROR’。
3. 返回 ‘L.elem[i-1]’（因为在物理结构中从第 0 号位开始存储）。

1.3.7 查找元素

函数名称：‘LocateElem(L, e)’
初始条件：线性表已存在
操作结果：返回 L 中第一个等于 ‘e’ 的数据元素的位置，若不存在这样的元素，

则返回 0 操作步骤:

1. 检查 'L.elem' 是否为 'NULL'，若不是，则返回 'ERROR'。
2. 从第一个元素开始遍历顺序表 L，直到最后一个元素。
3. 若当前元素等于 'e'，则返回其位置。
4. 若遍历完整个顺序表 L 仍未找到 'e'，则返回 0。

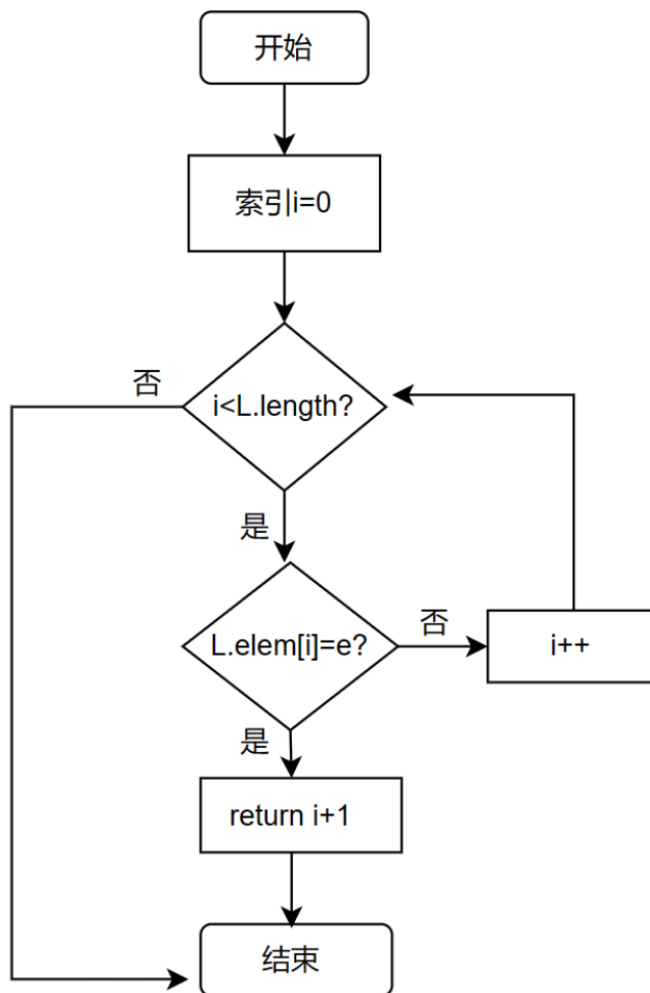


图 1-1 查找元素流程图

1.3.8 获得前驱

函数名称: 'PriorElem(L, cur_e, pre_e)'

初始条件: 线性表 L 已存在

操作结果: 若 'cur_e' 是 L 的数据元素且不是第一个元素，则用 'pre_e' 返回它的前驱，否则操作失败，'pre_e' 无定义操作步骤:

1. 检查 'L.elem' 是否为 'NULL'，若不是，则返回 'ERROR'。
2. 从第二个元素开始遍历顺序表 L，直到最后一个元素。
3. 若当前元素是 'cur_e'，则返回其前驱，即 'L.elem[i-1]'。

4. 若遍历完整个顺序表 L 仍未找到 'cur_e'，则操作失败。

1.3.9 获得后继

函数名称：'NextElem(L, cur_e, next_e)' 初始条件：线性表 L 已存在

操作结果：若 'cur_e' 是 L 的数据元素且不是最后一个元素，则用 'next_e' 返回它的后继，否则操作失败，'next_e' 无定义

操作步骤：

1. 检查 'L.elem' 是否为 'NULL'，若不是，则返回 'ERROR'。
2. 从前向后遍历顺序表，若遇到与 'cur_e' 相等的元素则停止，并记录当前位置 'pos'。
3. 若 'pos' 是最后一个元素的位置，则返回 'ERROR'，否则返回 'L.elem[pos+1]'。

1.3.10 插入元素

函数名称：'ListInsert(L, i, e)'

初始条件：线性表 L 已存在， $1 \leq i \leq \text{ListLength}(L) + 1$ 操作结果：在 L 的第 'i' 个位置之前插入新的数据元素 'e' 操作步骤：

1. 检查 'L.elem' 是否为 'NULL'，若不是，则返回 'ERROR'。
2. 判断顺序表是否已满，若已满，则需要扩容。
3. 判断插入位置 'i' 是否合法，即 $1 \leq i \leq \text{ListLength}(L) + 1$ ，若不合法，则操作失败。
4. 将第 'i' 位置及其后的所有元素后移一位。
5. 将新元素 'e' 插入到第 'i' 位置。

算法 1.1. ListInsert (顺序表插入)

Input: 顺序表 L , 插入位置 i , 元素 e

Output: 插入是否成功

```
if  $L$  不存在 then
    return INFEASIBLE
end if
检查内存分配是否成功
for  $j = L.length$  downto  $i$  do
     $L.elem[j] \leftarrow L.elem[j - 1]$ 
end for
将元素插入到位置  $i$ :  $L.elem[i - 1] \leftarrow e$ 
return OK
```

1.3.11 删除元素

函数名称为 ListDelete(L, i, e); 初始条件是线性表 L 已存在且非空, $1 \leq i \leq \text{ListLength}(L)$; 操作结果是删除 L 中第 i 个数据元素, 并用 e 返回其值; 操作步骤如下:

1. 检查 $L.elem$ 是否为 NULL, 如果不是, 返回 ERROR。
2. 确认删除位置 i 是否有效, 即 $1 \leq i \leq \text{ListLength}(L)$ 。若无效, 则操作失败。
3. 将第 i 位置及其后续元素前移一位。
4. 将删除的元素值赋给 e 。

详细操作将由下面的流程图给出:

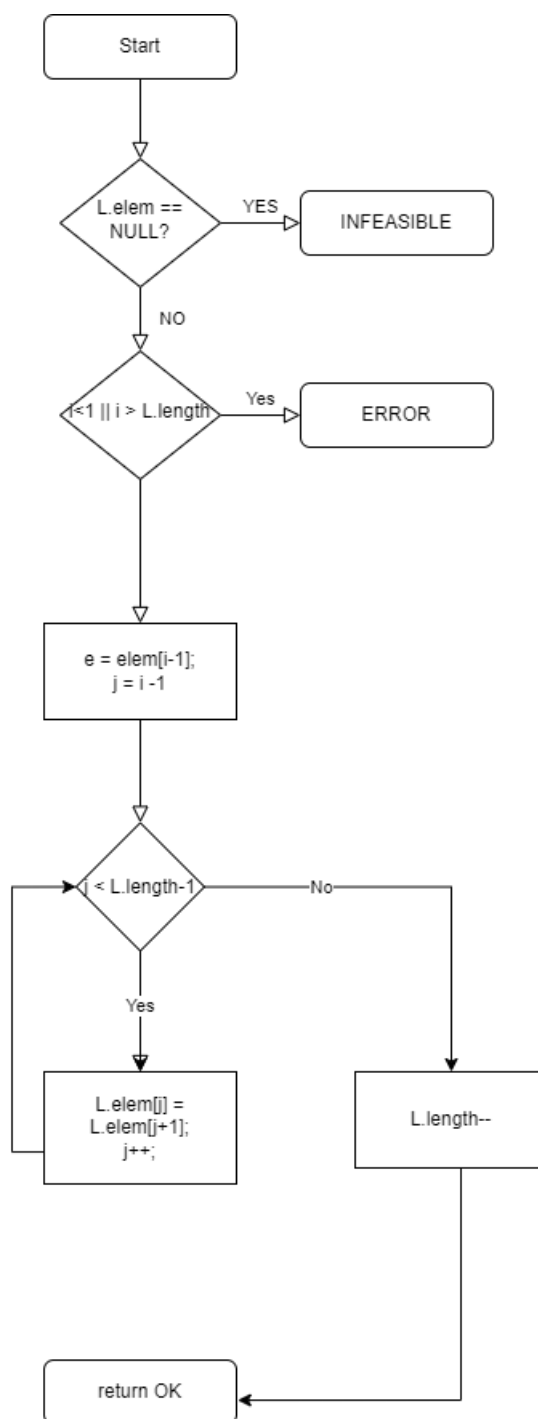


图 1-2 线性表删除元素程序流程图

1.3.12 遍历表

函数名称为 ListTraverse(L,visit()), 初始条件是线性表 L 已存在; 操作结果是依次对 L 的每个数据元素调用函数 visit()。操作步骤如下:

1. 检查 L.elem 是否为 NULL, 若是, 则返回 ERROR。
2. 遍历 L, 依次对每个数据元素调用 visit() 函数。

1.3.13 最大连续字数和

函数名称为 `MaxSubArray(L)`；初始条件是线性表 `L` 已存在且非空，请找出具有最大和的连续子数组（子数组最少包含一个元素），操作结果是返回最大和；实现思路如下：可采用动态规划方法解决此问题，设状态 F_i 为以第 i 个元素结尾的最大连续子数组和，其状态转移方程为：

$$F_i = \max(F_{i-1} + a_i, a_i)$$

复杂度为 $O(n)$ 。以下是伪代码：

```
Initialization:  $F_1 = a_1, MAX = -\infty$ ;  
for  $2 \leq i \leq L.length$  do  
     $F_i = \max\{F_{i-1} + a_i, a_i\}$ ;  
end for  
for  $1 \leq i \leq L.length$  do;  
    if  $F_i > MAX$  then;  
         $MAX = F_i$ ;  
    end if  
end for  
return  $MAX$ ;
```

1.3.14 和为 k 的子数组

函数名称为 `SubArrayNum(L,k)`；初始条件是线性表 `L` 已存在且非空；操作结果是找出该数组中和为 k 的连续子数组的个数。实现思路如下：

首先，建立顺序表的前缀和数组 `pre[0...L.length]`，其中 `pre[0]` 初始为 0，建立方法是从头开始遍历线性表，使得 $pre[i] = pre[i-1] + a[i]$ 。

将计数变量 `cnt` 初始化为 0，使用 i 和 j 两层循环遍历 `pre` 数组，其中 $i, j \in [0, L.length]$ ，如果 $i < j$ 且 $pre[j] - pre[i] = k$ ，则 $cnt = cnt + 1$ 。

最后返回 `cnt`。

1.3.15 顺序表排序

函数名称为 `sortList(L)`；初始条件是线性表 `L` 已存在；操作结果是将 `L` 从小到大排序；排序方法为冒泡排序。思路如下伪代码表示：

算法 1.2. Bubble Sort

Input: List L with n elements

Output: Sorted list L

```
procedure BubbleSort(L)
  for i = 0 to n - 1 do
    for j = 0 to n - 1 - i do
      if L[j] > L[j + 1] then
        Swap L[j] and L[j + 1]
      end if
    end for
  end for
```

1.3.16 实现线性表的文件形式保存

函数名称为 LoadList(L,FileName); 初始条件是线性表 L 已存在且非空; 操作结果是读取文件内容, 创建一个线性表。实现思路如下:

从文件中逐个读取元素, 用插入新元素的方法插入线性表, 读到 EOF 时程序终止。

函数名称为 SaveList(L,FileName); 初始条件是线性表 L 已存在; 操作结果是将线性表中数据存入指定文件。实现思路如下:

从头到尾遍历线性表, 将元素逐个打印进文件。

1.3.17 实现多个线性表管理

设计相应的数据结构管理多个线性表的查找、添加、移除等功能。

定义结构体后, 定义 Delete 函数和 locate 函数用于切换不同表格。Deletelist 接受 ListVec 和 char* 两个参数后, 寻找 arr 中同名顺序表并将其后名称整体前移, length-。Locatelist 则是寻找 arr 中同名表并返回位序, 主函数切换到该线性表。我认为我的亮点之处在于我将多个线性表管理与初级系统合在了一起, 在初始化顺序表的同时也初始化了线性表, 是系统更加简洁且完善。

1.3.17.1 部分函数实现方法:

1.ListInsert (顺序表插入) 该函数需判断插入位置是否合法, 若空间不足则动态扩容, 然后将插入位置及其后的元素依次后移, 最后插入新元素。

算法 1.1. ListInsert (顺序表插入)

Input: 顺序表 L , 插入位置 i , 元素 e

Output: 插入是否成功

if L 不存在 **then**

return INFEASIBLE

end if

检查内存分配是否成功

for $j = L.length$ **downto** i **do**

$L.elem[j] \leftarrow L.elem[j - 1]$

end for

将元素插入到位置 i : $L.elem[i - 1] \leftarrow e$

return OK

华中科技大学课程实验报告

2. **MaxSubArray**（最大连续子数组和） 采用动态规划思想，遍历数组，记录当前子数组和与最大值。

算法 1.3. MaxSubArray（最大连续子数组和）

Input: 顺序表 L

Output: 最大连续子数组和 $maxSum$

$maxSum \leftarrow L.elem[0]$

$currentSum \leftarrow 0$

for $i = 0$ **to** $L.length - 1$ **do**

if $currentSum > 0$ **then**

$currentSum \leftarrow currentSum + L.elem[i]$

else

$currentSum \leftarrow L.elem[i]$

end if

if $currentSum > maxSum$ **then**

$maxSum \leftarrow currentSum$

end if

end for

return $maxSum$

华中科技大学课程实验报告

3. **AddList**（多顺序表添加） 支持批量添加，需判断名称唯一性，动态分配空间，
并循环插入元素直到输入 0 结束。

算法 1.3. AddList（多顺序表添加）

Input: 多顺序表 *Lists*，新表名称 *ListName*

Output: 添加是否成功

```
for 每个待添加顺序表 do
    while 名称重复 do
        提示重新输入 ListName
    end while
    if 顺序表数量已达上限 then
        return ERROR
    end if
    分配新表空间，初始化
    while 输入元素  $e \neq 0$  do
        插入  $e$  到新表
    end while
    添加成功
end for
return OK
```

1.4 系统测试

该部分将展示一个实际的测试，先从文件读入一个线性表，并且依次测试各函数，会涉及到各函数的正常反应和异常处理，文件中 in.txt 的数据为

$\{31, -41, 59, 26, -53\}$

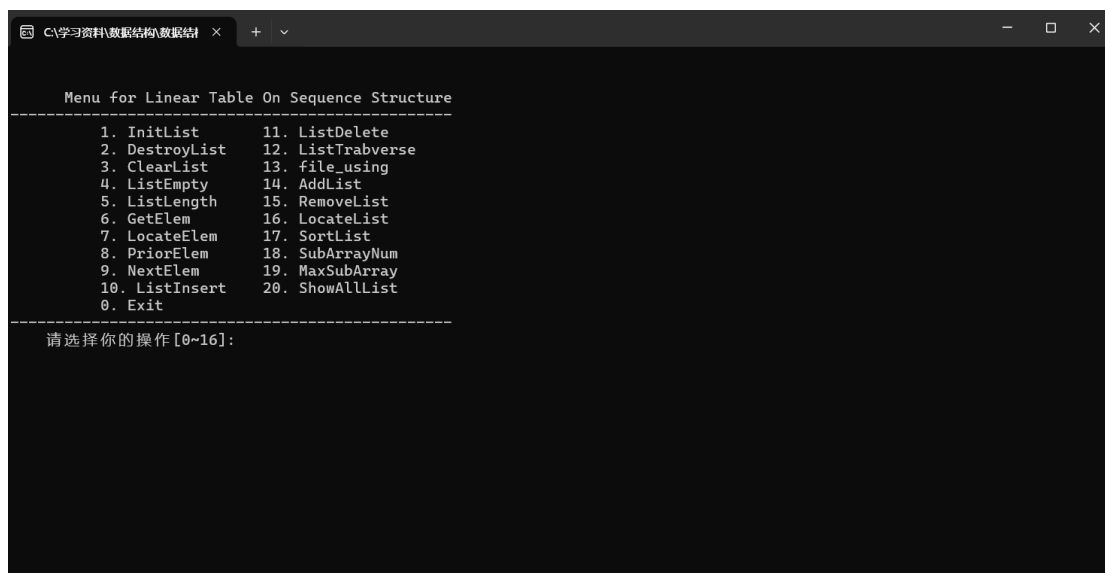
函数名	测试输入	输出	此时内部状态
InitList	/	初始化成功！	新建了一个线性表， 并且 L.length=0
ListEmpty	/	线性表为空！	此时 L.length=0 返回为空

华中科技大学课程实验报告

LoadList	in.txt	读取失败！	由于线性表已存在，无法读取
DestroyList	/	销毁成功！	释放 L.elem 对应空间，并令 L.elem=NULL
ListEmpty	/	线性表不存在！	此时 L.elem=NULL 报错
LoadList	in.txt	读取成功！	线性表内数据读取完成
ListEmpty	/	线性表不是空的！	此时 L.length=5 返回不是空的
ListLength	/	线性表长度为 5！	返回 L.length
GetElem	0	输入的序号不合法！	由于 i<1, 序号不合法报错
GetElem	3	第 3 个元素值为 59！	则返回第 3 位置的值
LocateElem	100	查找失败！	表中不存在 100，返回查找失败
LocateElem	26	该元素的序号是 4！	26 在表中位序为 4
PriorElem	31	查找失败！	31 为第一个元素，无前驱
PriorElem	59	元素的前驱是 -41	58 前驱为-41
NextElem	100	查找失败	表中不存在 100
NextElem	26	该元素后继为 -53	26 后继为-53
ListInsert	0 100	插入失败！	位序不合法报错
ListInsert	1 100	插入成功！	此时表中元素为 100, 31, -41, 59, 26, -53
ListDelete	12	删除失败！	表长 6 小于输入的 12, 报错
ListDelete	1	删除成功！	此时表中元素为 31,-41,59,26, -53
ListTraverse	/	{31, -41, 59, 26, -53}	遍历完成

华中科技大学课程实验报告

SaveList	out.txt	保存成功！	此时产生了一个名为out.txt 文件，内容为该数组
CreateNewList	newList	创建成功！	创建了名为 newList 的新表
LocateList	newList	切换成功！	切换到了 newList
ListEmpty	/	线性表不存在！	因为 newList 还未初始化
LocateList	0	切换成功！	切换到了初始表
RemoveList	newList	删除成功！	删除了 newList
LocateList	newList	切换失败！	由于已被删除，无法切换



```
C:\学习资料\数据结构\数据结 x + -

Menu for Linear Table On Sequence Structure
-----
1. InitList      11. ListDelete
2. DestroyList  12. ListTraverse
3. ClearList    13. file_using
4. ListEmpty    14. AddList
5. ListLength   15. RemoveList
6. GetElem      16. LocateList
7. LocateElem   17. SortList
8. PriorElem    18. SubArrayNum
9. NextElem     19. MaxSubArray
10. ListInsert  20. ShowAllList
0. Exit
-----

请选择你的操作 [0~16]:
```

操作界面演示

1.5 实验小结

这次实验中，我遇到了许多困难，特别是一些细节出现各种错误。譬如创建链表时要检查是否已存在，求前驱后继时要检验是否为首/尾元素，还有是一些需要算法的功能，例如求最大子数组和需要用到动态规划的思想等等。我通过查阅资料，询问同学等方式解决了此问题，并收获颇丰。

实验 2 基于邻接表的图实现

2.1 问题描述

采用邻接表作为图的物理结构，实现图的基本运算，从而加深对图的概念、基本运算的理解；熟练掌握图的逻辑结构与物理结构的关系。

2.2 系统设计

系统通过一个大的 while 循环实现菜单选择功能。每次进入循环时，使用变量 op 来读取用户的选择，根据 op 的值使用 switch 函数跳转到相应的功能函数并执行对应操作。当 op 为 0 时，退出循环。

进入循环后，系统会打印数字 0-20 对应的功能菜单，并提示用户输入 0-20 范围内的数字。以下是各选项对应的功能函数：

1. 创建图 (CreateCraph)
2. 销毁图 (DestroyGraph)
3. 查找顶点 (LocateVex)
4. 顶点赋值 (PutVex)
5. 获得第一邻接点 (FirstAdjVex)
6. 获得下一邻接点 (NextAdjVex)
7. 插入顶点 (InsertVex)
8. 删除顶点 (DeleteVex)
9. 插入弧 (InsertArc)
10. 删除弧 (DeleteArc)
11. 深度优先搜索遍历 (DFS_Traverse)
12. 广度优先搜索遍历 (BFS_Traverse)
13. 距离小于 k 的顶点集合 (VerticesSetLessThanK)
14. 顶点间最短路径和长度 (ShortestPathLength)
15. 图的连通分量 (ConnectedComponentsNums)
16. 实现图的文件形式保存

2.2.1 数据结构和一些常量的定义

```
#include "stdio.h"
#include "stdlib.h"
#include <string.h>
#include <queue>
#include <vector>
using namespace std;
```

华中科技大学课程实验报告

```
#define TRUE 1
#define FALSE 0
#define OK 1
#define ERROR 0
#define INFEASIBLE -1
#define OVERFLOW -2
#define MAX_VERTEX_NUM 20
typedef int status;
typedef int KeyType;
typedef enum
{
    DG,
    DN,
    UDG,
    UDN
} GraphKind;
typedef struct
{
    KeyType key;
    char others[20];
} VertexType; // 顶点类型定义

typedef struct ArcNode
{
    // 表结点类型定义
    int adjvex; // 顶点位置编号
    struct ArcNode *nextarc; // 下一个表结点指针
} ArcNode;
typedef struct VNode
{
    // 头结点及其数组类型定义
    VertexType data; // 顶点信息
    ArcNode *firstarc; // 指向第一条弧
} VNode, AdjList[MAX_VERTEX_NUM];
typedef struct
{
    // 邻接表的类型定义
    AdjList vertices; // 头结点数组
    int vexnum, arcnum; // 顶点数、弧数
    GraphKind kind; // 图的类型
} ALGraph;

typedef struct
```

```
{ // 线性表的管理表定义
    struct
    {
        char name[30];
        ALGraph T;
    } elem[10];
    int length;
    int listsize;
} LISTS;
```

2.3 系统实现

2.3.1 创建图

本函数 CreateGraph(G, V, VR) 用于根据顶点集 V 和关系集 VR 构造图 G 。以下为详细操作步骤:

1. 初始化顶点数组 pos 为 -1，用以记录每个顶点关键字的位置。
2. 逐个读入顶点：
 - 如果顶点数超限或顶点关键字重复，返回 ERROR。
 - 否则，将顶点信息存入 $G.vertices[vexnum]$ 并更新 pos 数组。
 - 顶点数 vexnum 自增。
3. 若图为空（即 $G.vexnum = 0$ ），返回 ERROR。
4. 逐个读入弧，并构建图结构：
 - 检查两端顶点存在性，若不存在，返回 ERROR。
 - 检查弧是否已存在，若已存在，跳过；否则，插入新弧并更新 arcnum。
5. 完成所有操作后返回 OK。

2.3.2 销毁图

函数名称是 DestroyGraph(G)；初始条件图 G 已存在；操作结果是销毁图 G ；以下为操作步骤：若 $G.vexnum == 0$ ，返回 ERROR。否则令 $vexnum = 0$ ，返回 OK。

2.3.3 查找顶点

本函数 LocateVex(G, u) 用于在图 G 中查找与给定关键字 u 相匹配的顶点位置。以下为详细操作步骤:

1. 初始化循环，使用索引 i 遍历图 G 的顶点数组 vertices。

2. 对每个顶点检查:

- 如果顶点的关键字等于 u , 即 $G.vertices[i].key == u$, 则返回 i , 表示找到了顶点的位置。

3. 如果遍历完所有顶点后未找到匹配的关键字, 返回 -1 , 表示顶点在图中不存在。该函数是图操作中的基本查询工具, 用于确定顶点在数组中的位置。

2.3.4 顶点赋值

本函数 $LocateVex(G, u)$ 用于在图 G 中查找与给定关键字 u 相匹配的顶点位置。以下为详细操作步骤:

1. 初始化变量 pos 为 -1 , 表示未找到匹配的顶点。
2. 遍历图 G 的所有顶点:
 - 使用变量 cur 作为循环变量。
 - 若顶点的关键字与 u 匹配, 则将 pos 设为该顶点的序号, 并退出循环。
3. 返回 pos , 若未找到匹配顶点, 则 pos 仍为 -1 , 表示顶点不存在。

2.3.5 顶点赋值

本函数 $PutVex(G, u, value)$ 用于在图 G 中查找顶点 u , 并将其值修改为 $value$ 。以下为详细操作步骤:

1. 创建一个大小为 100 的整型数组 pos , 初始值均为 -1 , 用于记录每个顶点关键字在 $vertices$ 数组中的位置。
2. 遍历图 G 的所有顶点, 将顶点的关键字作为索引存入 pos 数组, 并将对应的位置值设为顶点在 $vertices$ 数组中的索引。
3. 检查顶点 u 是否存在及关键字是否唯一:
 - 如果 $pos[u]$ 为 -1 (即顶点 u 不存在) 或 u 不等于 $value.key$ 并且 $pos[value.key]$ 不等于 -1 (即关键字不唯一), 返回 $ERROR$ 。
4. 将 $G.vertices[pos[u]].data$ 修改为 $value$ 。
5. 返回 OK 。

2.3.6 获得第一邻接点

本函数 $FirstAdjVex(G, u)$ 用于在图 G 中查找顶点 u 的第一邻接顶点。以下为详细操作步骤:

1. 初始化变量 `pos` 为 `-1`，表示未找到匹配的顶点。
2. 遍历图 `G` 的所有顶点：
 - 使用变量 `i` 作为循环变量。
 - 若顶点的关键字与 `u` 匹配，则将 `pos` 设为该顶点的序号，并退出循环。
3. 检查 `pos` 是否为 `-1`：
 - 如果是，则返回 `-1`，表示顶点 `u` 不存在。
4. 检查顶点 `u` 是否有第一邻接顶点：
 - 如果存在，返回 `G.vertices[pos].firstarc->adjvex`。
 - 如果不存在，返回 `-1`。

2.3.7 获得下一邻接点

本函数 `NextAdjVex(G, v, w)` 用于在图 `G` 中查找顶点 `v` 的下一个邻接顶点。
以下为详细操作步骤：

1. 初始化变量 `pos` 为 `-1`，表示未找到匹配的顶点 `v`。
2. 遍历图 `G` 的所有顶点：
 - 使用变量 `i` 作为循环变量。
 - 若顶点的关键字与 `v` 匹配，则将 `pos` 设为该顶点的序号，并退出循环。
3. 如果 `pos` 为 `-1`，返回 `-1`，表示顶点 `v` 不存在。
4. 获取顶点 `v` 的邻接表头指针 `tmp`。
5. 遍历顶点 `v` 的邻接表，找到顶点 `w`：
 - 若顶点 `w` 的关键字与 `w` 匹配，则检查其下一个邻接顶点。
 - 如果 `w` 的下一个邻接顶点存在，返回其位序。
6. 如果没有找到匹配的邻接顶点或没有下一个邻接顶点，返回 `-1`。

2.3.8 插入顶点

本函数 `InsertVex(G, v)` 用于在图 `G` 中插入顶点 `v`。以下为详细操作步骤：

1. 检查图 `G` 的顶点数是否已达到最大值 `MAX_VERTEX_NUM`，如果是则返回 `ERROR`。
2. 创建一个大小为 `100` 的整型数组 `flag`，初始值均为 `0`，用于记录每个顶点关键字是否已存在。
3. 遍历图 `G` 的所有顶点，将顶点的关键字作为索引存入 `flag` 数组，并将对应的位置值设为 `1`。
4. 检查顶点 `v` 的关键字是否已存在：

- 如果 `flag[v.key]` 为 1, 返回 ERROR。
- 5. 将顶点 `v` 插入图 `G` 的顶点数组:
 - 将 `G.vertices[G.vexnum].data` 设为 `v`。
 - 将 `G.vertices[G.vexnum].firstarc` 设为 NULL。
- 6. 增加图 `G` 的顶点数 `vexnum`。
- 7. 返回 OK。

2.3.9 删除顶点

本函数 `DeleteVex(G, v)` 用于在图 `G` 中删除顶点 `v`。以下为详细操作步骤:

1. 检查图 `G` 的顶点数是否为 1, 如果是则返回 ERROR。
2. 创建一个大小为 100 的整型数组 `pos`, 初始值均为 -1, 用于记录每个顶点关键字在 `vertices` 数组中的位置。
3. 遍历图 `G` 的所有顶点, 将顶点的关键字作为索引存入 `pos` 数组, 并将对应的位置值设为顶点在 `vertices` 数组中的索引。
4. 检查顶点 `v` 是否存在:
 - 如果 `pos[v]` 为 -1, 返回 ERROR。
5. 删除顶点 `v` 的所有邻接边:
 - 遍历顶点 `v` 的邻接表, 并删除每条边, 同时更新相关顶点的邻接表。
 - 减少图 `G` 的边数 `arcnum`。
6. 删除顶点 `v` 并移动顶点数组以填补删除顶点后的空缺。
7. 更新所有顶点的邻接表中指向的顶点索引, 以反映顶点数组的变化。
8. 减少图 `G` 的顶点数 `vexnum`。
9. 返回 OK。

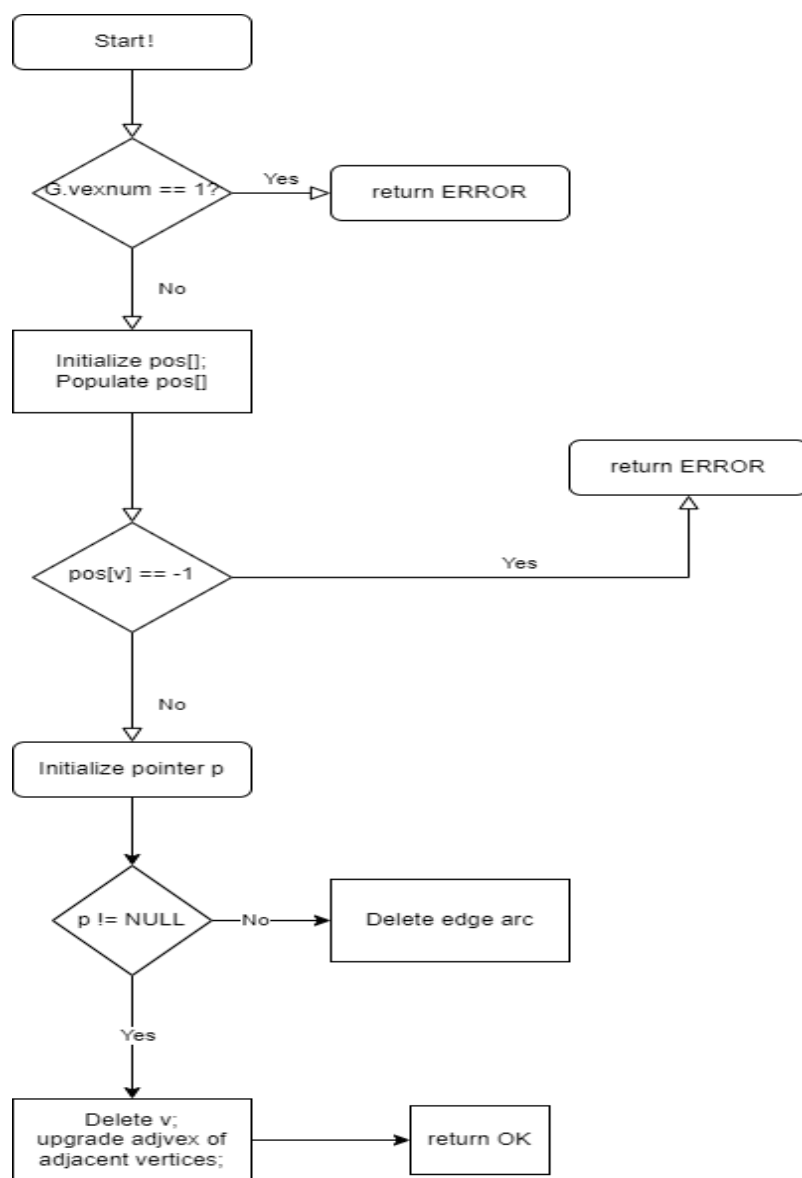


图 2-1 删除顶点流程图

2.3.10 插入弧

本函数 `InsertArc(G, v, w)` 用于在图 `G` 中插入弧 `vw`。以下为详细操作步骤：

1. 创建一个大小为 100 的整型数组 `pos`，初始值均为 -1，用于记录每个顶点关键字在 `vertices` 数组中的位置。
2. 遍历图 `G` 的所有顶点，将顶点的关键字作为索引存入 `pos` 数组，并将对应的位置值设为顶点在 `vertices` 数组中的索引。
3. 检查顶点 `v` 和 `w` 是否存在：

- 如果 $\text{pos}[v]$ 或 $\text{pos}[w]$ 为 -1, 返回 ERROR。
- 4. 检查弧 vw 是否已存在:
 - 遍历顶点 v 的邻接表, 如果找到邻接顶点为 w 的弧, 则返回 ERROR。
- 5. 创建新的弧节点 newArcv 和 newArcw :
将 newArcv 插入顶点 v 的邻接表。
 - 将 newArcw 插入顶点 w 的邻接表。
- 6. 增加图 G 的弧数 arcnum 。
- 7. 返回 OK。

2.3.11 删除弧

本函数 $\text{DeleteArc}(G, v, w)$ 用于在图 G 中删除弧 $\langle v, w \rangle$ 。以下为详细操作步骤:

1. 创建一个大小为 100 的整型数组 pos , 初始值均为 -1, 用于记录每个顶点关键字在 vertices 数组中的位置。
2. 遍历图 G 的所有顶点, 将顶点的关键字作为索引存入 pos 数组, 并将对应的位置值设为顶点在 vertices 数组中的索引。
3. 检查顶点 v 和 w 是否存在:
 - 如果 $\text{pos}[v]$ 或 $\text{pos}[w]$ 为 -1, 返回 ERROR。
4. 检查弧 $\langle v, w \rangle$ 是否存在:
 - 遍历顶点 v 的邻接表, 如果找到邻接顶点为 w 的弧, 则标记 flag 为 1 并退出循环。
 - 如果未找到, 返回 ERROR。
5. 删除弧 $\langle v, w \rangle$:
 - 遍历顶点 v 的邻接表, 找到指向 w 的弧并删除。
6. 删除弧 $\langle w, v \rangle$:
 - 遍历顶点 w 的邻接表, 找到指向 v 的弧并删除。
7. 减少图 G 的弧数 arcnum 。
8. 返回 OK。

2.3.12 DFS

本函数 $\text{DFS}(G, \text{visit})$ 用于对图 G 进行深度优先搜索遍历, 依次对图中的每一个顶点使用函数 visit 访问一次, 仅访问一次。以下为详细操作步骤:

1. 初始化一个大小为 100 的布尔数组 visited , 用于记录每个顶点是否被访问

过。

2. 遍历图 G 的所有顶点，对每个顶点调用辅助函数 `dfs` 进行深度优先搜索。
3. 在 `dfs` 辅助函数中，若当前顶点未被访问，则调用 `visit` 函数访问该顶点，并标记为已访问。
4. 递归调用 `dfs` 函数对所有未访问的邻接顶点进行访问。

5.3.13 BFS

本函数 `BFSTraverse(G, visit)` 用于对图 G 进行广度优先搜索遍历，依次对图中的每一个顶点使用函数 `visit` 访问一次，仅访问一次。以下为详细操作步骤：

1. 定义一个大小为 100 的布尔数组 `visited`，用于记录每个顶点是否被访问过。
2. 初始化 `visited` 数组的所有值为 `false`。
3. 定义一个大小为 100 的整型数组 `queue`，用于存储待访问的顶点索引。
4. 初始化队列的左指针 `l` 和右指针 `r` 均为 0。
5. 对图 G 中的每一个顶点进行遍历：
 - 如果该顶点已被访问过，则跳过本次循环。
 - 将该顶点标记为已访问。
 - 将该顶点的索引加入队列中。
 - 当队列不为空时，继续进行广度优先搜索遍历：
 - 获取当前队列的左指针 `l` 所指向的顶点索引 `cur`。
 - 调用 `visit` 函数访问当前顶点。
 - 将队列的左指针 `l` 向右移动一位。
 - 获取当前顶点的第一个弧头节点，并遍历所有邻接节点：
 - *如果该弧尾节点未被访问过，将其标记为已访问，并将其索引加入队列中。
6. 返回 `OK`。

算法 2.1. Breadth-First Search Traversal

Input: Graph G , Function `visit` to access each vertex

Output: Traversal result of graph G

```
procedure BFSTraverse(G, visit)
    Initialize boolean array visited[100] to false
    Initialize integer array queue[100]
    Initialize integers l and r to 0
    for i = 0 to G.vexnum - 1 do
        if visited[i] then
            continue
```

```
end if
visited[i] = true
queue[r] = i
r = (r + 1) % 100
while l ≠ r do
    cur = queue[l]
    visit(G.vertices[cur].data)
    l = (l + 1) % 100
    tmp = G.vertices[cur].firstarc
    while tmp ≠ NULL do
        if visited[tmp.adjvex] = false then
            visited[tmp.adjvex] = true
            queue[r] = tmp.adjvex
            r = (r + 1) % 100
        end if
        tmp = tmp.nextarc
    end while
end while
end for
return OK
end procedure=0
```

2.3.14 距离小于 K 的顶点集合

本函数 VerticesSetLessThanK(G, e, k, visit) 用于对距离顶点 e 小于 k 的所有顶点使用函数 visit 访问一次。以下为详细操作步骤：

1. 调用 LocateVex(G, e) 函数定位顶点 e 在图 G 中的位置：
 - 如果顶点 e 不存在，直接返回。
2. 初始化布尔数组 visited，用于记录每个顶点是否被访问过。
3. 初始化队列 queue，用于存储待访问的顶点索引。
4. 初始化变量：
 - l 和 r 为队列的左右指针。
 - cnt 为当前层的顶点数，初始值为 1。
 - nextLevel 为下一层的顶点数。
5. 将顶点 e 的索引加入队列，并标记为已访问。
6. 进行 k 层广度优先搜索：
 - 对当前层的所有顶点进行访问：
 - 访问当前顶点 cur 并调用 visit 函数。

- 遍历当前顶点 `cur` 的所有邻接顶点：
 - *对于未访问的邻接顶点，将其加入队列，并标记为已访问。
 - *更新下一层的顶点数 `nextLevel`。
- 更新当前层的顶点数 `cnt` 为 `nextLevel`。

2.3.15 顶点间最短路径长度

本函数 `ShortestPathLength` 用于计算顶点 `u` 到顶点 `v` 的最短路径长度。如果顶点 `u` 和 `v` 不连通，则返回 `INF`。以下为详细操作步骤：

1. 使用 `LocateVex` 函数查找顶点 `u` 和 `v` 的位置。
2. 如果顶点 `u` 或 `v` 不存在，返回 `-1`。
3. 初始化访问数组 `visited` 为 `false`。
4. 初始化队列 `queue`，队列指针 `l` 和 `r`，以及当前层节点数 `cnt` 和下一层节点数 `nextLevel`。
5. 将顶点 `u` 加入队列并标记为已访问。
6. 执行广度优先搜索，直到找到顶点 `v` 或队列为空：
 - 遍历当前层的所有节点，如果找到顶点 `v`，则设置标志 `flag` 并跳出循环。
 - 将当前层的邻接节点加入队列并标记为已访问。
 - 更新下一层的节点数 `nextLevel`。
7. 如果找到顶点 `v`，返回路径长度 `len`；否则返回 `INF` 表示顶点 `u` 和 `v` 不连通。

2.3.16 图的连通分量

本函数 `ConnectedComponentsNums` 用于计算图 `G` 中的连通块数量。以下为详细操作步骤：

1. 初始化连通块数量 `cnt` 为 `0`。
2. 初始化访问数组 `visited` 为 `false`。
3. 初始化队列 `queue`，队列指针 `l` 和 `r` 均为 `0`。
4. 遍历图 `G` 的所有顶点：
 - 如果顶点已被访问，跳过本次循环。
 - 发现新的连通块，增加 `cnt`。
 - 将顶点标记为已访问，并将其加入队列。
 - 当队列不为空时，继续进行广度优先搜索遍历：
 - 获取当前队列的左指针 `l` 所指向的顶点索引 `cur`。

- 将队列的左指针 1 向右移动一位。
- 获取当前顶点的第一个弧头节点，并遍历所有邻接节点：
 - *如果邻接顶点未被访问，将其标记为已访问，并将其加入队列。

5. 返回连通块数量 cnt。

2.3.17 图文件形式保存和读取

1. 本函数 SaveGraph(G, FileName) 用于将图 G 的数据保存到文件 FileName 中。以下为详细操作步骤：

1. 定义一个整型数组 stack，用于存储顶点的邻接顶点。
2. 初始化栈顶指针 top 为 0。
3. 打开文件 FileName 进行写操作，如果打开失败，返回 ERROR。
4. 遍历图 G 的所有顶点：
 - 将顶点数据写入文件，包括顶点的关键字和其他信息。
 - 重置栈顶指针 top 为 0。
 - 获取顶点的第一个弧节点，并将顶点的邻接顶点入栈。
 - 将栈中的邻接顶点写入文件。
 - 写入结束标志 -1 表示该顶点的邻接顶点结束。
5. 关闭文件。
6. 返回 OK。

2. 本函数 LoadGraph(G, FileName) 用于从文件 FileName 中读取图 G 的数据并创建图的邻接表。以下为详细操作步骤：

1. 如果图中已有顶点或弧，返回 ERROR。
2. 打开文件 FileName，如果打开失败，返回 ERROR。
3. 初始化关键变量：
 - key: 顶点关键字
 - adj: 邻接顶点索引
 - name: 顶点其他信息
 - tmp: 临时变量
 - cntVex: 顶点计数
 - cntArc: 弧计数
4. 读取文件内容，创建图的邻接表：
 - 读取顶点的关键字和其他信息。
 - 复制顶点的其他信息。
 - 初始化顶点的弧链表。

- 读取邻接顶点并创建弧节点，加入顶点的弧链表中。
 - 增加弧计数。
 - 增加顶点计数。
5. 更新图的顶点数和弧数。
 6. 关闭文件。
 7. 返回 OK。

2.4 系统测试

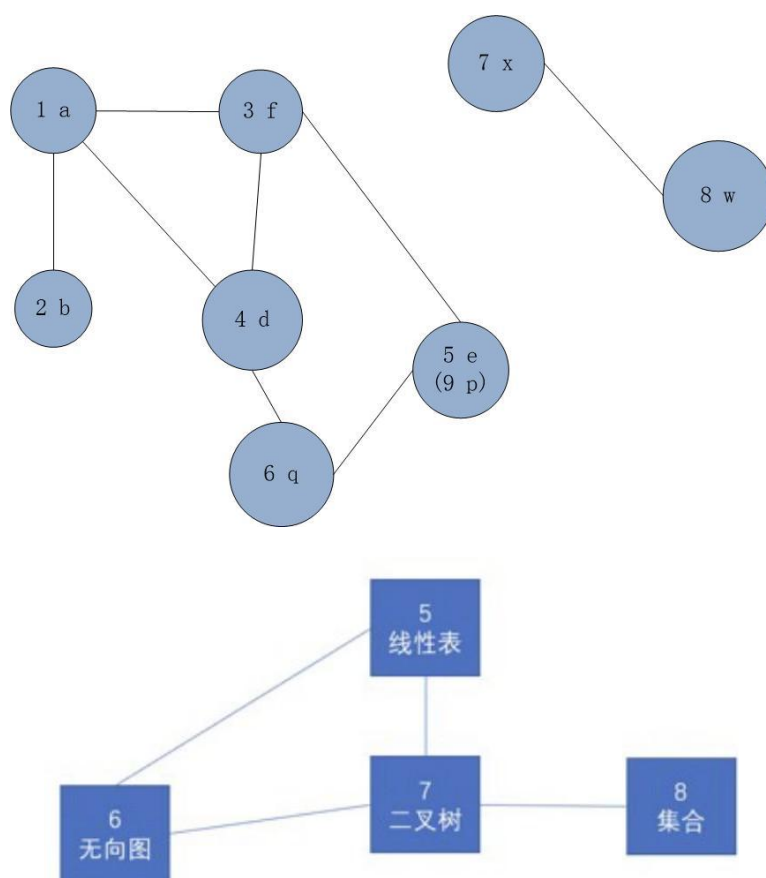


图2-2 测试数据

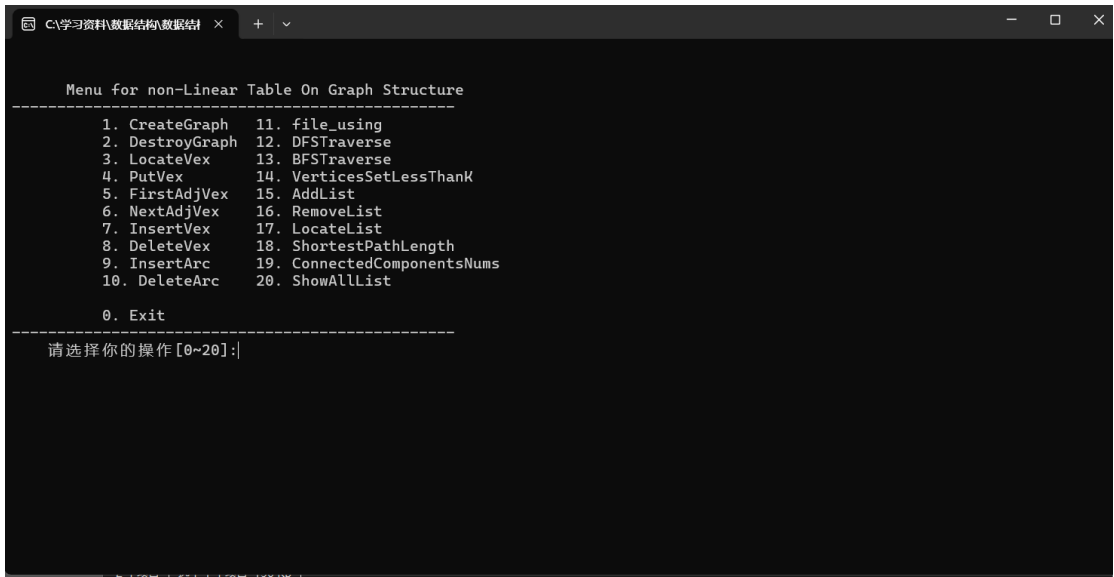


图 2-3 操作演示界面

表 2-1 测试数据

函数名	测试输入	输出	此时内部状态
InitGraph	5 线性表 8 集合 7 二叉树 6 无向图 -1 nil 5 6 5 7 6 7 7 8 -1 -1	初始化成功！	新建了一个如图所示的图
InsertArc	7 1	插入失败！	关键字为 1 的顶点不存在，报错
DFSTraverse	/	5 线性表 7 二叉树 8 集合 6 无向图	完成深度优先遍历
BFSTraverse	/	5 线性表 7 二叉树 6 无向图 8 集合	完成广度优先遍历

华中科技大学课程实验报告

DeleteArc	7 8	删除成功!	删除了 7-8 之间的边
Connected Components Nums	/	2	删除 78 之间边之后图断为两块
PutVex	5 5 数据结构	赋值成功!	5 线性表被重命名为 5 数据结构
PutVex	6 5 a	赋值失败!	关键字 5 已被占用
DeleteVex	5	删除成功!	5 数据结构被删除
DFSTraverse	/	6无向图,7 二叉树,8 集合	完成深度优先遍历
LocateVex	6	该顶点位序是 0	删去 5 后其它顶点位序整体前移, 6 移到第 0 位
InsertVex	5 线性表	插入成功!	插入新节点
InsertArc	5 6	插入成功!	连接 56
InsertArc	5 7	插入成功!	连接 57
InsertArc	7 8	插入成功!	连接 78
SaveGraph	a.txt	存储成功!	已将邻接表存储到 a.txt 中
CreateGraph	newGraph	创建成功!	新创建了名为 newGraph 的图
LocateGraph	newGraph	切换成功!	切换到了 newGraph
LoadGraph	a.txt	读取成功!	读取文件
DFSTraverse	/	6 无向图 5 线性表 7 二叉树 8 集合	完成深度优先遍历
Connected Components Nums	/	1	整个图连通, 只有一个连通块
ShortestPath Length	6 8	2	68 之间最短距离为 2

华中科技大学课程实验报告

DeleteArc	7 8	删除成功!	删除了 7-8 之间的边
VerticesSet LessThanK	6 2	6 无向图 5 线性 表 7 二叉树	以 6 为中心两单位 为半径内部只有 5 6 7 三节点
LocateGraph	0	切换成功!	切换到了初始图
DeleteGraph	newGraph	删除成功!	删除了 newGraph

2.5 实验小结

本次实验实现了一个用邻接表存储的图及相应操作。在任务过程中,我认为有许多难点,譬如创建图时对空图的判断,删除顶点时要删除相应的弧等等操作,要十分小心和细心。实现额外附加功能时也通过询问同学,与同学探讨,查找资料等方法学习并成功实现了附加功能,收获颇丰。

3 课程的收获和建议

通过本次课程，大大加强了我调试程序，查找编程错误的能力，为将来写更大的项目作了铺垫。此外，加深了我对数据结构中线性表、链表、树、图等内容的理解，并让我初步接触了一些算法的思想譬如动态规划等等。让我的编程能力大大增加，同时我也希望能为这个课程提供一些建议，首先就是有时候下大雨的时候，希望老师能够让学生们转为线上授课，这样不仅缓解学生对考勤的压力，老师们也可以节约通勤时间，进而为科研提供更多的时间。

3.1 基于顺序存储结构的线性表实现

通过本次实验，我在理论认知和实践能力方面获得了以下深刻收获：

1. 顺序存储结构的底层实现机制：通过手动实现动态扩容功能，深入理解了顺序表在内存中的连续存储特性。具体实现了当当前存储空间不足时，以原容量 1.5 倍进行扩容的策略，并验证了这种扩容方式在均摊分析下可以达到 $O(1)$ 的插

入时间复杂度。在实现过程中，通过对比 `malloc/realloc` 等内存操作函数的不同使用场景，掌握了内存管理的核心要点。

2. 复杂边界条件的处理经验：在实现 `ListInsert` 和 `ListDelete` 函数时，系统性

地考虑了多种边界情况：包括表满扩容时的内存分配失败、插入位置为表首/表尾的特殊处理、删除唯一元素后表的维护等。通过编写针对性的测试用例，如连续插入 10000 个元素测试扩容稳定性、交替进行头插和尾插测试位置计算正确性等，培养了严谨的工程思维。

3. 多表管理系统的设计能力：创新性地实现了支持最多 10 个顺序表同时管理的 `LISTS` 结构。通过设计表名唯一性校验机制、表索引快速定位等功能，深入理解了资源管理的核心思想。在实现过程中，解决了表删除后索引维护、跨表操作等关键技术难点，最终系统可以稳定支持表的创建、删除、切换和批量操作。

4. 文件持久化的完整实现：开发了完善的序列化方案，将顺序表结构（包括元素数据、当前长度、容量等信息）通过二进制方式保存到文件。在实现过程中，解决了字节对齐、数据校验等关键问题，并设计了配套 `loadListFromFile` 函数，可以准确还原表状态。通过对比文本和二进制两种存储方式的性能差异，深入理解了 I/O 优化的基本原则。

5. 算法实践的综合提升：在完成基础功能后，实现了 `MaxSubArray` 等扩展

算法。通过将动态规划思想应用于实际数据结构，加深了对算法优化的理解。特别是在实现过程中，通过逐步优化从 $O(n^3)$ 到 $O(n)$ 的不同版本，直观体会了算法改进带来的性能提升。

3.2 基于链式存储结构的线性表实现

在本次实现过程中，得益于老师们对知识脉络的梳理以及助教的点拨，我不仅扎实掌握了链表的原理与实践，更深刻理解了链式存储结构的精髓及其适用场景，我认为这极大提升了我在实际问题中选择最优数据结构的能力。同时，在老师们构建的清晰框架和助教们的实操引导下，我熟练掌握了线性表的核心操作（插入、删除、查找等），并成功在链式结构中实现了这些操作的高效版本。这次学习体验充分展现了课程设计的深度与实用性。

3.3 基于二叉链表的二叉树实现

在本次实现过程中，得益于老师们引人入胜的教学和助教的思维引导，我对二叉树的核心要义（定义、性质、遍历）获得了更深的了解。通过老师们构建的清晰知识图谱和助教提供的帮助，我不仅熟练运用二叉链表实现了二叉树结构，更明白其在处理复杂树形数据时的独特价值。而且，老师们对递归、分治等核心范式富有洞见的阐释，配合助教在实战演练中的思维纠偏，让我增强了解决相关问题的效率与代码可维护性。

3.4 基于邻接表的图实现

1. 邻接表结构的深度掌握：通过手动实现顶点表+ 边链表的存储结构，深入理解了邻接表在表示稀疏图时的空间优势。在实现过程中，解决了边节点内存管理、顶点快速定位等关键技术问题。特别在实现头插法建立邻接表时，通过维护多个辅助指针，确保了边插入的高效性和正确性。

2. 图遍历算法的完整实现：采用递归和非递归两种方式实现了 DFS 算法，并通过对比实验验证了递归深度限制问题。在 BFS 实现中，使用队列结构保证了层次遍历的正确性，同时设计了 visited 数组防止重复访问。通过实际测试不同规模的图数据（包括链状图、完全图等特殊结构），验证了算法的时间复杂度理论。

3. 复杂图操作的系统实践：完整实现了包括顶点删除、边插入等复杂操作。特别是在 DeleteVex 函数中，开发了“先删除关联边再移除顶点”的两阶段处理机制，确保了数据一致性。通过设计特殊的测试用例（如删除中心顶点、删除孤立顶点等），验证了各种边界情况下的处理正确性。

4. 最短路径算法的优化实现：基于 BFS 实现了无权图最短路径算法，通过维护 distance 数组记录路径长度。在实现过程中，优化了传统的 BFS 算法，使其在找到目标顶点时立即返回，减少了不必要的计算。通过测试网格图、树状图等多种拓扑结构，验证了算法的正确性和效率。

5. 图可视化的调试技术：开发了基于控制台的简易可视化功能，可以将图结构以文本方式直观展示。在调试过程中，通过分析顶点间的连接关系，快速定位了多个边指针维护的错误。这种调试方法显著提高了复杂指针操作的开发效率。

6. 多图管理系统的工程实践：设计实现了支持图切换、图持久化的管理系统。通过将图结构与元信息（名称、创建时间等）分离存储，实现了灵活的多图管理。系统支持图的跨会话保存和加载，通过二进制文件格式确保了数据完整性，文件头包含校验和等安全机制。

通过本课程的系统学习，我不仅掌握了数据结构的核心实现技术，更重要的是培养了系统级的编程思维和工程实现能力。特别是在处理复杂指针操作、内存管理和算法优化等方面获得了突破性进步，这些经验将直接助力后续的算法学习和项目开发。

通过本课程的系统学习，我不仅掌握了数据结构的核心实现技术，更重要的是培养了系统级的编程思维和工程实现能力。特别是在处理复杂指针操作、内存管理和算法优化等方面获得了突破性进步，这些经验将直接助力后续的算法学习和项目开发。

参考文献

- 1 庞丽萍,阳富民著. 计算机操作系统(第2版). 北京:人民邮电出版社,2014.
- 2 [美] Andrew S.Tanenbaum, Herbert Bos 著,陈向群,马洪兵译. 现代操作系统(原书第4版). 北京:机械工业出版社,2017.
- 3 [德] Michael Kerrisk 著,孙剑,徐从年等译. Linux/UNIX 系统编程手册. 北京:人民邮电出版社,2014.
- 4 [美] Robert Love 著,陈莉君,康华译. Linux 内核设计与实现(原书第3版). 北京:机械工业出版社,2011.
- 5 [美] Randal E.Bryant, David O'Hallaron 著,龚奕利,贺莲译. 深入理解计算机系统(原书第3版). 北京:机械工业出版社,2016.
- 6 Arch Linux 官方 wiki 文档. URL: <https://wiki.archlinux.org/>
- 7 严蔚敏等.数据结构(C语言版).清华大学出版社
- 8 Larry Nyhoff. ADTs, Data Structures, and Problem Solving with C++. Second Edition,Calvin College,2005
- 9 殷立峰.Qt,C++跨平台图形界面程序设计基础.清华大学出版社,2014:192~197
- 10 严蔚敏等.数据结构题集(C语言版).清华大学出版社

附录 A 基于顺序存储结构线性表实现的源程序

```
#include <stdio.h>
#include <stdlib.h>
#include <windows.h>
#include <string.h>
// 宏定义和类型定义 (来自 def.h)
#define TRUE 1
#define FALSE 0
#define OK 1
#define ERROR 0
#define INFEASIBLE -1
#define OVERFLOW -2
typedef int status;
typedef int ElemType; // 数据元素类型定义
#define LIST_INIT_SIZE 100
#define LISTINCREMENT 10
typedef struct
{ // 顺序表 (顺序结构) 的定义
    ElemType *elem;
    int length;
    int listsize;
} SqList;
typedef struct
{ // 线性表的管理表定义
    struct
    {
        char name[30];
        SqList L;
    } elem[10];
    int length;
    int listsize;
} LISTS;
status SaveList(SqList &L, char FileName[]);
status LoadList(SqList &L, char FileName[]);
void AddList(LISTS &Lists, char ListName[]);
status PriorElem(SqList L, ElemType e, ElemType &pre);
status NextElem(SqList L, ElemType e, ElemType &next);
int LocateList(LISTS Lists, char ListName[]);
status ListTraverse(SqList L);
```

华中科技大学课程实验报告

```
status RemoveList(LISTS &Lists, char ListName[]);
int LocateElem(Sqlist L, ElemType e);
status ListLength(Sqlist L);
status ListInsert(Sqlist &L, int i, ElemType e);
status ListEmpty(Sqlist L);
status ListDelete(Sqlist &L, int i, ElemType &e);
status GetElem(Sqlist L, int i, ElemType &e);
status DestroyList(Sqlist &L);
status ClearList(Sqlist &L);
int sortList(Sqlist &L);
void SubArrayNum(Sqlist L);
void MaxSubArray(Sqlist L);
void swap(int *a, int *b);
int medianOfThree(int arr[], int low, int high);
int partition(int arr[], int low, int high);
void quickSort(int arr[], int low, int high); // 快速排序
int main()
{
    SetConsoleOutputCP(CP_UTF8);
    LISTS Lists;
    Lists.length = 0;
    int op = 1;
    int list_num = 1;
    int count = 0;
    while (op)
    {
        char name_temp[30];
        printf("\n\n");
        printf("      Menu for Linear Table On Sequence Structure \n");
        printf("-----\n");
        if (count > 1)
        {
            printf("需要查看有什么表吗? ,需要请输入1,不需要输入其他数字\n");
            scanf("%d", &op);
            if (op == 1)
            {
                printf("创建的线性表有: \n");
                for (int temp_2 = 0; temp_2 < Lists.length; temp_2++)
                {
                    printf("%s\n", Lists.elem[temp_2].name);
                }
            }
        }
    }
}
```

华中科技大学课程实验报告

```
    }
}
bool flag = 1;
printf("请问你想对哪个线性表进行操作? \n");
do
{
    scanf("%s", name_temp);
    if (!(list_num = LocateList(Lists, name_temp)))
    {
        flag = 0;
        printf("找不到对应的线性表,请重新输入\n");
    }
    else
        flag = 1;
} while (flag != OK);
}
printf("      1. InitList      11. ListDelete\n");
printf("      2. DestroyList   12. ListTraverse\n");
printf("      3. ClearList      13. file_using\n");
printf("      4. ListEmpty      14. AddList\n");
printf("      5. ListLength     15. RemoveList\n");
printf("      6. GetElem        16. LocateList\n");
printf("      7. LocateElem     17. SortList\n");
printf("      8. PriorElem      18. SubArrayNum\n");
printf("      9. NextElem       19. MaxSubArray\n");
printf("     10. ListInsert     20. ShowAllList\n");
printf("      0. Exit\n");
printf("-----\n");
printf("    请选择你的操作[0~16]:");
scanf("%d", &op);
switch (op)
{
case 1: // 线性表的创建
    if (Lists.length > 0)
    {
        printf("已初始化线性表\n");
        getchar();
        getchar();
        break;
    }
}
```


华中科技大学课程实验报告

```
printf("请输入你想创建的线性表名称\n");
char firstname[30];
scanf("%s", firstname);
AddList(Lists, firstname);
printf("请输入元素,以 0 结束\n");
int i;
scanf("%d", &i);
while (i)
{
    ListInsert(Lists.elem[List.length - 1].L, Lists.elem[L
ists.length - 1].L.length + 1, i);
    scanf("%d", &i);
}
if (!ListTraverse(Lists.elem[List.length - 1].L))
    printf("线性表是空表! \n");
else
    count++;
getchar();
getchar();
break;
case 2: // 销毁线性表
    if (DestroyList(Lists.elem[list_num - 1].L) == OK)
        printf("销毁线性表成功\n");
    else
        printf("线性表已空\n");
    getchar();
    getchar();
    break;
case 3: // 清空线性表
    if (ClearList(Lists.elem[list_num - 1].L) == OK)
        printf("清空线性表成功\n");
    else
        printf("线性表已空\n");
    getchar();
    getchar();
    break;
case 4: // 线性表判空
    if (ListEmpty(Lists.elem[list_num - 1].L) == OK)
        printf("线性表为空\n");
    else if (ListEmpty(Lists.elem[list_num - 1].L) == 0)
```

华中科技大学课程实验报告

```
        printf("线性表不为空\n");
    else
        printf("线性表不存在\n");
    getchar();
    getchar();
    break;
case 5: // 线性表长度
    if (ListLength(Lists.elem[list_num - 1].L) > 0)
        printf("线性表长度
为%d\n", ListLength(Lists.elem[list_num - 1].L));
    else
        printf("线性表为空\n");
    getchar();
    getchar();
    break;
case 6: // 获取元素
    int temp, m;
    printf("您想获取第几个元素? \n");
    scanf("%d", &i);
    if (GetElem(Lists.elem[list_num - 1].L, m, temp) == OK)
        printf("第%d 个元素是:%d\n", m, temp);
    else
        printf("i 的范围不正确\n");
    getchar();
    getchar();
    break;
case 7: // 查找元素
    printf("您想查找哪个元素? \n");
    scanf("%d", &i);
    if ((temp = LocateElem(Lists.elem[list_num - 1].L, i)) > 0)
    {
        printf("元素%d 在第%d 位\n", i, temp);
    }
    else
        printf("不存在该元素\n");
    getchar();
    getchar();
    break;
case 8: // 获取前驱元素
    int pre;
```

华中科技大学课程实验报告

```
printf("您想查找哪个元素的前驱? \n");
scanf("%d", &i);
if (PriorElem(Lists.elem[list_num - 1].L, i, pre) == OK)
    printf("该元素的前驱是%d\n", pre);
else
    printf("不存在该元素\n");
getchar();
getchar();
break;
case 9: // 获取后继元素
    int next;
    printf("您想查找哪个元素的后驱? \n");
    scanf("%d", &i);
    if (NextElem(Lists.elem[list_num - 1].L, i, next) == OK)
        printf("该元素的后驱是%d\n", next);
    else
        printf("不存在该元素\n");
    getchar();
    getchar();
    break;
case 10: // 插入元素
    printf("请输入要插入的元素\n");
    scanf("%d", &temp);
    printf("请输入要插入的位置\n");
    scanf("%d", &m);
    if (ListInsert(Lists.elem[list_num - 1].L, m, temp) == OK)
    {
        printf("插入成功\n");
        printf("插入后的线性表为:\n");
        if (!ListTraverse(Lists.elem[list_num - 1].L))
            printf("线性表是空表! \n");
    }
    else
        printf("插入失败\n");
    getchar();
    getchar();
    break;
case 11: // 删除元素
    printf("请输入删除的位置\n");
    scanf("%d", &temp);
```

华中科技大学课程实验报告

```
        if (ListDelete(Lists.elem[list_num - 1].L, temp, m) == OK)
        {
            printf("删除成功\n");
            printf("删除后的线性表为:");
            if (!ListTraverse(Lists.elem[list_num - 1].L))
                printf("空表! \n");
        }
        else
            printf("删除失败\n");
        getchar();
        getchar();
        break;
    case 12: // 遍历线性表
        if (!ListTraverse(Lists.elem[list_num - 1].L))
            printf("线性表是空表! \n");
        getchar();
        getchar();
        break;
    case 13: // 线性表读写文件
        char filename[30];
        printf("请输入指令\n");
        printf("1. 文件读取\n");
        printf("2. 文件写入\n");
        scanf("%d", &i);
        if (i == 1)
        {
            printf("请输入需要读取的文件地址, 请用.dat 作为后缀\n");
            scanf("%s", filename);
            printf("请输入你想创建的线性表名称\n");
            scanf("%s", firstname);
            AddList(Lists, firstname);
            if (LoadList(Lists.elem[List.length - 1].L, filename)
== OK)
            {
                printf("读取成功\n");
                count++;
            }
            else
                printf("读取失败\n");
        }
    }
```

华中科技大学课程实验报告

```
        else if (i == 2)
        {
            printf("请输入需要写入的文件地址,请用.dat 作为后缀\n");
            scanf("%s", filename);
            if (SaveList(Lists.elem[list_num - 1].L, filename) == OK)
                printf("写入成功\n");
            else
                printf("写入失败\n");
        }
        getchar();
        getchar();
        break;
case 14: // 多线性表管理: 增加一个新线性表
    printf("请输入新线性表名字\n");
    scanf("%s", firstname);
    if ((temp = LocateList(Lists, firstname)) > 0)
    {
        printf("有重复名称,无法创建\n");
        getchar();
        getchar();
        break;
    }
    AddList(Lists, firstname);
    printf("请输入元素,以 0 结束\n");
    scanf("%d", &i);
    while (i)
    {
        ListInsert(Lists.elem[List.length - 1].L, Lists.elem[List.length - 1].L.length + 1, i);
        scanf("%d", &i);
    }
    if (!ListTraverse(Lists.elem[List.length - 1].L))
        printf("线性表是空表! \n");
    else
        count++;
    getchar();
    getchar();
    break;
case 15: // 多线性表管理: 移除一个线性表
    printf("请输入需要移除的线性表名字\n");
```

华中科技大学课程实验报告

```
scanf("%s", firstname);
if (RemoveList(Lists, firstname) == OK)
    printf("移除成功\n");
else
    printf("移除失败\n");
getchar();
getchar();
break;
case 16: // 多线性表管理: 查找线性表
    printf("请输入需要查找的线性表名字\n");
    scanf("%s", firstname);
    if ((temp = LocateList(Lists, firstname)) == 0)
        printf("查找失败\n");
    else
        printf("查找成功,位置为%d\n", temp);
    getchar();
    getchar();
    break;
case 17:
    temp = sortList(Lists.elem[list_num - 1].L);
    if (temp)
    {
        printf("排序成功,排序后的线性表为:");
        for (int i = 0; i < Lists.elem[list_num - 1].L.length; i++)
        {
            printf("%d ", Lists.elem[list_num - 1].L.elem[i]);
        }
    }
    getchar();
    getchar();
    break;
case 18:
    SubArrayNum(Lists.elem[list_num - 1].L);
    getchar();
    getchar();
    break;
case 19:
    MaxSubArray(Lists.elem[list_num - 1].L);
    getchar();
    getchar();
```

华中科技大学课程实验报告

```
        break;
    case 20:
        printf("创建的线性表有: \n");
        for (int temp_2 = 0; temp_2 < Lists.length; temp_2++)
        {
            printf("%s\n", Lists.elem[temp_2].name);
        }
    case 0:
        break;
    } // end of switch
} // end of while
printf("欢迎下次再使用本系统! \n");
return 0;
} // end of main()

status SaveList(SqlList &L, char FileName[])
// 如果线性表L 存在, 将线性表L 的元素写到FileName 文件中, 返回OK, 否则返回
INFEASIBLE。
{
    if (L.elem == NULL)
        return -1;
    else
    {
        FILE *fp = fopen(FileName, "wb");
        fwrite(&L.length, sizeof(ElemType), 1, fp);
        fwrite(L.elem, sizeof(ElemType), L.length, fp);
        fclose(fp);
        return 1;
    }
}

status LoadList(SqlList &L, char FileName[])
// 如果线性表L 不存在, 将FileName 文件中的数据读入到线性表L 中, 返回OK, 否则返回
INFEASIBLE。
{
    if (L.length != 0)
        return -1;
    else
    {
        FILE *fp = fopen(FileName, "rb");
        fread(&L.length, sizeof(ElemType), 1, fp);
        L.elem = (ElemType *)malloc(sizeof(ElemType) * LIST_INIT_SIZE);
```

华中科技大学课程实验报告

```
        fread(L.elem, sizeof(ElemType), L.length, fp);
        fclose(fp);
        return 1;
    }
}

void AddList(LISTS &Lists, char ListName[])
{
    SqList L;
    Lists.elem[Lists.length++].L.elem = (ElemType *)malloc(sizeof(ElemT
ype) * LIST_INIT_SIZE);
    Lists.elem[Lists.length - 1].L.length = 0;
    Lists.elem[Lists.length - 1].L.listsize = LIST_INIT_SIZE;
    strcpy(Lists.elem[Lists.length - 1].name, ListName);
}

status PriorElem(SqList L, ElemType e, ElemType &pre)
{
    if (L.elem == NULL)
        return -1;
    else
    {
        for (int i = 0; i < L.length; i++)
        {
            if (L.elem[i] == e && i > 0)
            {
                pre = L.elem[i - 1];
                return 1;
            }
        }
        return 0;
    }
}

status NextElem(SqList L, ElemType e, ElemType &next)
{
    if (L.elem == NULL)
        return -1;
    else
    {
        for (int i = 0; i < L.length; i++)
        {
            if (L.elem[i] == e && i < L.length - 1)
```



```
        {
            next = L.elem[i + 1];
            return 1;
        }
    }
    return 0;
}

int LocateList(LISTS Lists, char ListName[])
{
    for (int i = 0; i < Lists.length; i++)
    {
        if (strcmp(Lists.elem[i].name, ListName) == 0)
            return i + 1;
    }
    return 0;
}

status ListTraverse(Sqlist L)
{
    if (L.elem == NULL)
        return -1;
    if (L.length == 0)
        return 1;
    else
    {
        int i;
        for (i = 0; i < L.length - 1; i++)
            printf("%d ", L.elem[i]);
        printf("%d", L.elem[i]);
        return 1;
    }
}

status RemoveList(LISTS &Lists, char ListName[])
{
    for (int i = 0; i < Lists.length; i++)
    {
        if (strcmp(Lists.elem[i].name, ListName) == 0)
        {
            for (int j = i; j < Lists.length; j++)
            {
```

```
        Lists.elem[j] = Lists.elem[j + 1];
    }
    Lists.length--;
    return 1;
}
}
return 0;
}
int LocateElem(SqList L, ElemType e)
{
    if (L.elem == NULL)
        return -1;
    else
    {
        for (int i = 0; i < L.length; i++)
        {
            if (L.elem[i] == e)
                return i + 1;
        }
        return 0;
    }
}
status ListLength(SqList L)
{
    if (L.elem == NULL)
        return -1;
    else
        return L.length;
}
#include <malloc.h>
status ListInsert(SqList &L, int i, ElemType e)
{
    if (i < 1 || i > L.length + 1)
        return 0;
    else if (L.elem == NULL)
        return -1;
    else
    {
        L.elem = (ElemType *)realloc(L.elem, sizeof(ElemType) * 11);
        int j;
```

```
        for (j = L.length - 1; j > i - 2; j--)
        {
            L.elem[j + 1] = L.elem[j];
        }
        L.elem[j + 1] = e;
        L.length++;
        return 1;
    }
}

status ListEmpty(SqList L)
{
    if (L.length == 0 && L.elem != NULL)
        return 1;
    else if (L.length != 0 && L.elem != NULL)
        return 0;
    else
        return -1;
}

status ListDelete(SqList &L, int i, ElemType &e)
{
    if (i < 1 || i >= L.length + 1)
        return 0;
    else if (L.elem == NULL)
        return -1;
    else
    {
        e = L.elem[i - 1];
        for (int j = i - 1; j < L.length; j++)
        {
            L.elem[j] = L.elem[j + 1];
        }
        L.length--;
        return 1;
    }
}

status GetElem(SqList L, int i, ElemType &e)
{
    if (L.elem == NULL)
        return -1;
    else if (i < 1 || i > L.length)
```

```
        return 0;
    else
    {
        e = L.elem[i - 1];
        return 1;
    }
}

status DestroyList(Sqlist &L)
{
    if (L.elem == NULL)
        return -1;
    else
    {
        L.elem = 0;
        free(L.elem);
        return 1;
    }
}

status ClearList(Sqlist &L)
{
    if (L.elem == NULL)
        return -1;
    else
    {
        L.length = 0;
        return 1;
    }
}

void swap(int *a, int *b);
int medianOfThree(int arr[], int low, int high);
int partition(int arr[], int low, int high);
void quickSort(int arr[], int low, int high); // 快速排序
int sortList(Sqlist &L)
{
    if (L.elem == NULL || L.length == 0)
    {
        printf("线性表为空\n");
        return 0;
    }
    quickSort(L.elem, 0, L.length - 1);
}
```

```
        return 1;
    }
    void swap(int *a, int *b)
    {
        int temp = *a;
        *a = *b;
        *b = temp;
    }
    int medianOfThree(int arr[], int low, int high)
    {
        int mid = low + (high - low) / 2;
        if (arr[low] > arr[mid])
            swap(&arr[low], &arr[mid]);
        if (arr[low] > arr[high])
            swap(&arr[low], &arr[high]);
        if (arr[mid] > arr[high])
            swap(&arr[mid], &arr[high]);
        return mid;
    }
    int partition(int arr[], int low, int high)
    {
        int pivotIndex = medianOfThree(arr, low, high);
        swap(&arr[pivotIndex], &arr[high]);
        int pivot = arr[high];
        int i = low - 1;
        for (int j = low; j < high; j++)
        {
            if (arr[j] < pivot)
            {
                i++;
                swap(&arr[i], &arr[j]);
            }
        }
        swap(&arr[i + 1], &arr[high]);
        return i + 1;
    }
    void quickSort(int arr[], int low, int high)
    {
        while (low < high)
        {
```

华中科技大学课程实验报告

```
int pi = partition(arr, low, high);
if (pi - low < high - pi)
{
    quickSort(arr, low, pi - 1);
    low = pi + 1;
}
else
{
    quickSort(arr, pi + 1, high);
    high = pi - 1;
}
}
}
void SubArrayNum(Sqlist L)
{
    if (L.elem == NULL || L.length == 0)
    {
        printf("线性表为空\n");
        return;
    }
    int k;
    printf("请输入你想求和的连续子数组 k\n");
    int result = 0;
    scanf("%d", &k);
    for (int i = 0; i < L.length; i++)
    {
        int count = 0;
        for (int j = i; j < L.length; j++)
        {
            count = count + L.elem[j];
            if (count == k)
            {
                result++;
            }
        }
    }
    printf("有%d个这样的子数组", result);
    return;
}
void MaxSubArray(Sqlist L)
```

```
{
    if (L.elem == NULL || L.length == 0)
    {
        printf("线性表为空\n");
        return;
    }
    int k;
    int count = 0;
    int max_count = 0;
    for (int i = 0; i < L.length; i++)
    {
        count = count + L.elem[i];
        if (count > max_count)
            max_count = count;
        if (count <= 0)
            count = 0;
    }
    printf("最大连续子数组和为:%d", max_count);
    return;
}
```

附录 B 基于链式存储结构线性表实现的源程序

```
#include "stdio.h"
#include "stdlib.h"
#include "string.h"
#include <windows.h>
#define TRUE 1
#define FALSE 0
#define OK 1
#define ERROR 0
#define INFEASIBLE -1
#define OVERFLOW -2
typedef int status;
typedef int ElemType;
#define LIST_INIT_SIZE 100
#define LISTINCREMENT 10
typedef struct LNode
{
    ElemType data;
    struct LNode *next;
} LNode, *LinkList;
typedef struct
{
    struct
    {
        char name[30];
        LinkList L;
    } elem[10];
    int length;
    int listsize;
} LISTS;
status SaveList(LinkList L, char FileName[]);
status LoadList(LinkList &L, char FileName[]);
void AddList(LISTS &Lists, char ListName[]);
status PriorElem(LinkList L, ElemType e, ElemType &pre);
status NextElem(LinkList L, ElemType e, ElemType &next);
status InitList(LinkList &L);
status ListTraverse(LinkList L);
int ListLength(LinkList L);
status ListInsert(LinkList &L, int i, ElemType e);
status ListEmpty(LinkList L);
```


华中科技大学课程实验报告

```
status ListDelete(LinkList &L, int i, ElemType &e);
status GetElem(LinkList L, int i, ElemType &e);
status DestroyList(LinkList &L);
status ClearList(LinkList &L);
int LocateList(LISTS Lists, char ListName[]);
status RemoveList(LISTS &Lists, char ListName[]);
status LocateElem(LinkList L, ElemType e);
int sortList(LinkList L);
void swap(int *a, int *b);
int medianOfThree(int arr[], int low, int high);
int partition(int arr[], int low, int high);
void quickSort(int arr[], int low, int high);
status reverseList(LinkList &L);
status RemoveNthFromEnd(LinkList &L, int n);
int main()
{
    SetConsoleOutputCP(CP_UTF8);
    LISTS Lists;
    Lists.length = 0;
    int op = 1;
    int list_num = 1;
    int count = 0;
    char firstname[30];
    LNode *p, *tail_2;
    while (op)
    {
        char name_temp[30];
        printf("\n\n");
        printf("      Menu for Linear Table On Sequence Structure \n");
        printf("-----\n");
        if (count > 1)
        {
            printf("需要查看有什么表吗? ,需要请输入1,不需要输入其他数字\n");
            scanf("%d", &op);
            if (op == 1)
            {
                printf("创建的线性表有: \n");
                for (int temp_2 = 0; temp_2 < Lists.length; temp_2++)
                {
                    printf("%s\n", Lists.elem[temp_2].name);
                }
            }
        }
    }
}
```

华中科技大学课程实验报告

```
    }
}
bool flag = 1;
printf("请问你想对哪个线性表进行操作? \n");
do
{
    scanf("%s", name_temp);
    if (!(list_num = LocateList(Lists, name_temp)))
    {
        flag = 0;
        printf("找不到对应的线性表,请重新输入\n");
    }
    else
        flag = 1;
} while (flag != OK);
}
printf("      1. InitList      11. ListDelete\n");
printf("      2. DestroyList   12. ListTraverse\n");
printf("      3. ClearList      13. file_using\n");
printf("      4. ListEmpty      14. AddList\n");
printf("      5. ListLength     15. RemoveList\n");
printf("      6. GetElem        16. LocateList\n");
printf("      7. LocateElem     17. SortList\n");
printf("      8. PriorElem      18. reverseList\n");
printf("      9. NextElem       19. RemoveNthFromEnd\n");
printf("     10. ListInsert     20. ShowAllList\n");
printf("      0. Exit\n");
printf("-----\n");
printf("    请选择你的操作[0~16]:");
scanf("%d", &op);
switch (op)
{
case 1: // 线性表的创建
    if (Lists.length > 0)
    {
        printf("已初始化线性表\n");
        getchar();
        getchar();
        break;
    }
}
```

华中科技大学课程实验报告

```
printf("请输入你想创建的线性表名称\n");
scanf("%s", firstname);
AddList(Lists, firstname);
printf("请输入元素,以 0 结束\n");
int i;
scanf("%d", &i);
tail_2 = Lists.elem[Lists.length - 1].L;
while (i)
{
    p = (LNode *)malloc(sizeof(LNode));
    p->data = i;
    tail_2->next = p;
    tail_2 = tail_2->next;
    scanf("%d", &i);
}
tail_2->next = NULL;
if (!ListTraverse(Lists.elem[Lists.length - 1].L))
    printf("线性表是空表! \n");
else
    count++;
getchar();
getchar();
break;
case 2: // 销毁线性表
    if (DestroyList(Lists.elem[list_num - 1].L) == OK)
        printf("销毁线性表成功\n");
    else
        printf("线性表已空\n");
    getchar();
    getchar();
    break;
case 3: // 清空线性表
    if (ClearList(Lists.elem[list_num - 1].L) == OK)
        printf("清空线性表成功\n");
    else
        printf("线性表已空\n");
    getchar();
    getchar();
    break;
case 4: // 线性表判空
```

华中科技大学课程实验报告

```
        if (ListEmpty(Lists.elem[list_num - 1].L) == OK)
            printf("线性表为空\n");
        else if (ListEmpty(Lists.elem[list_num - 1].L) == 0)
            printf("线性表不为空\n");
        else
            printf("线性表不存在\n");
        getchar();
        getchar();
        break;
case 5: // 线性表长度
        if (ListLength(Lists.elem[list_num - 1].L) > 0)
            printf("线性表长度
为%d\n", ListLength(Lists.elem[list_num - 1].L));
        else
            printf("线性表为空\n");
        getchar();
        getchar();
        break;
case 6: // 获取元素
        int temp, m;
        printf("您想获取第几个元素? \n");
        scanf("%d", &i);
        if (GetElem(Lists.elem[list_num - 1].L, m, temp) == OK)
            printf("第%d 个元素是:%d\n", m, temp);
        else
            printf("i 的范围不正确\n");
        getchar();
        getchar();
        break;
case 7: // 查找元素
        printf("您想查找哪个元素? \n");
        scanf("%d", &i);
        if ((temp = LocateElem(Lists.elem[list_num - 1].L, i)) > 0)
        {
            printf("元素%d 在第%d 位\n", i, temp);
        }
        else
            printf("不存在该元素\n");
        getchar();
        getchar();
```

华中科技大学课程实验报告

```
        break;
    case 8: // 获取前驱元素
        int pre;
        printf("您想查找哪个元素的前驱? \n");
        scanf("%d", &i);
        if (PriorElem(Lists.elem[list_num - 1].L, i, pre) == OK)
            printf("该元素的前驱是%d\n", pre);
        else
            printf("不存在该元素\n");
        getchar();
        getchar();
        break;
    case 9: // 获取后继元素
        int next;
        printf("您想查找哪个元素的后驱? \n");
        scanf("%d", &i);
        if (NextElem(Lists.elem[list_num - 1].L, i, next) == OK)
            printf("该元素的后驱是%d\n", next);
        else
            printf("不存在该元素\n");
        getchar();
        getchar();
        break;
    case 10: // 插入元素
        printf("请输入要插入的元素\n");
        scanf("%d", &temp);
        printf("请输入要插入的位置\n");
        scanf("%d", &m);
        if (ListInsert(Lists.elem[list_num - 1].L, m, temp) == OK)
        {
            printf("插入成功\n");
            printf("插入后的线性表为:\n");
            if (!ListTraverse(Lists.elem[list_num - 1].L))
                printf("线性表是空表! \n");
        }
        else
            printf("插入失败\n");
        getchar();
        getchar();
        break;
```

华中科技大学课程实验报告

```
case 11: // 删除元素
    printf("请输入删除的位置\n");
    scanf("%d", &temp);
    if (ListDelete(Lists.elem[list_num - 1].L, temp, m) == OK)
    {
        printf("删除成功\n");
        printf("删除后的线性表为:");
        if (!ListTraverse(Lists.elem[list_num - 1].L))
            printf("空表! \n");
    }
    else
        printf("删除失败\n");
    getchar();
    getchar();
    break;
case 12: // 遍历线性表
    if (!ListTraverse(Lists.elem[list_num - 1].L))
        printf("线性表是空表! \n");
    getchar();
    getchar();
    break;
case 13: // 线性表读写文件
    char filename[30];
    printf("请输入指令\n");
    printf("1. 文件读取\n");
    printf("2. 文件写入\n");
    scanf("%d", &i);
    if (i == 1)
    {
        printf("请输入需要读取的文件地址, 请用.dat 作为后缀\n");
        scanf("%s", filename);
        printf("请输入你想创建的线性表名称\n");
        scanf("%s", firstname);
        if ((temp = LocateList(Lists, firstname)) > 0)
        {
            printf("有重复名称, 无法创建\n");
            break;
        }
        AddList(Lists, firstname);
    }
```

华中科技大学课程实验报告

```
        if (LoadList(Lists.elem[Lists.length - 1].L, filename)
== OK)
        {
            printf("读取成功\n");
            count++;
        }
        else
            printf("读取失败\n");
    }
    else if (i == 2)
    {
        printf("请输入需要写入的文件地址,请用.dat 作为后缀\n");
        scanf("%s", filename);
        if (SaveList(Lists.elem[list_num - 1].L, filename) == OK)
            printf("写入成功\n");
        else
            printf("写入失败\n");
    }
    getchar();
    getchar();
    break;
case 14: // 多线性表管理: 增加一个新线性表
    printf("请输入新线性表名字\n");
    scanf("%s", firstname);
    if ((temp = LocateList(Lists, firstname)) > 0)
    {
        printf("有重复名称,无法创建\n");
        break;
    }
    AddList(Lists, firstname);
    printf("请输入元素,以 0 结束\n");
    tail_2 = Lists.elem[Lists.length - 1].L;
    scanf("%d", &i);
    while (i)
    {
        p = (LNode *)malloc(sizeof(LNode));
        p->data = i;
        tail_2->next = p;
        tail_2 = tail_2->next;
        scanf("%d", &i);
    }
```

华中科技大学课程实验报告

```
    }
    tail_2->next = NULL;
    if (!ListTraverse(Lists.elem[Lists.length - 1].L))
        printf("线性表是空表! \n");
    else
        count++;
    getchar();
    getchar();
    break;
case 15: // 多线性表管理: 移除一个线性表
    printf("请输入需要移除的线性表名字\n");
    scanf("%s", firstname);
    if (RemoveList(Lists, firstname) == OK)
        printf("移除成功\n");
    else
        printf("移除失败\n");
    getchar();
    getchar();
    break;
case 16: // 多线性表管理: 查找线性表
    printf("请输入需要查找的线性表名字\n");
    scanf("%s", firstname);
    if ((temp = LocateList(Lists, firstname)) == 0)
        printf("查找失败\n");
    else
        printf("查找成功,位置为%d\n", temp);
    getchar();
    getchar();
    break;
case 17:
    temp = sortList(Lists.elem[list_num - 1].L);
    if (temp)
    {
        printf("排序成功,排序后的线性表为:");
        temp = ListTraverse(Lists.elem[list_num - 1].L);
    }
    getchar();
    getchar();
    break;
case 18:
```


华中科技大学课程实验报告

```
        if ((temp = reverseList(Lists.elem[list_num - 1].L)))
        {
            printf("反转成功,反转后的线性表为: ");
            temp = ListTraverse(Lists.elem[list_num - 1].L);
        }
        else if (temp == 0)
            printf("长度为 1,无需反转\n");
        getchar();
        getchar();
        break;
    case 19:
        printf("请输入你想删除链表的倒数第几个结点\n");
        scanf("%d", &i);
        temp = RemoveNthFromEnd(Lists.elem[list_num - 1].L, i);
        if (temp)
        {
            printf("删除成功,删除后的线性表为:\n");
            temp = ListTraverse(Lists.elem[list_num - 1].L);
        }
        else
            printf("删除失败\n");
        getchar();
        getchar();
        break;
    case 20:
        printf("创建的线性表有: ");
        for (int temp_2 = 0; temp_2 < Lists.length; temp_2++)
        {
            if (!strcmp(Lists.elem[temp_2].name, 0))
                printf("%s\n", Lists.elem[temp_2].name);
        }
        getchar();
        getchar();
        break;
    case 0:
        break;
    } // end of switch
} // end of while
printf("欢迎下次再使用本系统! \n");
return 0;
```

华中科技大学课程实验报告

```
} // end of main()
// file_using.h
status SaveList(LinkList L, char FileName[])
{
    if (L == NULL)
        return -1;
    FILE *fp = fopen(FileName, "wb");
    LNode *p = L;
    int count = -1;
    while (p)
    {
        p = p->next;
        count++;
    }
    fwrite(&count, sizeof(ElemType), 1, fp);
    p = L->next;
    for (int i = 0; i < count; i++, p = p->next)
        fwrite(&p->data, sizeof(ElemType), 1, fp);
    fclose(fp);
    return 1;
}

status LoadList(LinkList &L, char FileName[])
{
    if (L->next != NULL)
        return -1;
    int count;
    L = (LinkList)malloc(sizeof(LNode));
    LNode *tail = L;
    LinkList p;
    FILE *fp = fopen(FileName, "rb");
    fread(&count, sizeof(ElemType), 1, fp);
    for (int i = 0; i < count; i++, p = p->next)
    {
        p = (LinkList)malloc(sizeof(LNode));
        fread(&p->data, sizeof(ElemType), 1, fp);
        tail->next = p;
        tail = tail->next;
    }
    tail->next = NULL;
    fclose(fp);
}
```

华中科技大学课程实验报告

```
    return 1;
}
// AddList.h
void AddList(LISTS &Lists, char ListName[])
{
    Lists.elem[Lists.length++].L = (LinkList)malloc(sizeof(LNode));
    Lists.elem[Lists.length - 1].L->next = NULL;
    strcpy(Lists.elem[Lists.length - 1].name, ListName);
}
// PriorElem.h
status PriorElem(LinkList L, ElemType e, ElemType &pre)
{
    if (L == NULL)
        return -1;
    L = L->next;
    while (L)
    {
        if (L->next != NULL && L->next->data == e)
        {
            pre = L->data;
            return 1;
        }
        L = L->next;
    }
    return 0;
}
// NextElem.h
status NextElem(LinkList L, ElemType e, ElemType &next)
{
    if (L == NULL)
        return -1;
    L = L->next;
    while (L)
    {
        if (L->next != NULL && L->data == e)
        {
            next = L->next->data;
            return 1;
        }
        L = L->next;
    }
}
```

```
    }
    return 0;
}
// InitList.h
status InitList(LinkList &L)
{
    if (L != NULL)
        return -1;
    else
    {
        L = (LinkList)malloc(sizeof(LNode));
        L->next = NULL;
        return 1;
    }
}
// ListTraverse.h
status ListTraverse(LinkList L)
{
    if (L == NULL)
        return -1;
    else if (L->next == NULL)
        return 1;
    LNode *p = L->next;
    while (p)
    {
        printf("%d ", p->data);
        p = p->next;
    }
    return 1;
}
// ListLength.h
int ListLength(LinkList L)
{
    if (L == NULL)
        return -1;
    else
    {
        LinkList p = L->next;
        int count = 0;
        while (p != NULL)
```

华中科技大学课程实验报告

```
        {
            p = p->next;
            count++;
        }
        return count;
    }
}

// ListInsert.h
status ListInsert(LinkList &L, int i, ElemType e)
{
    if (L == NULL)
        return -1;
    LNode *p = L;
    int count = 0;
    while (p)
    {
        p = p->next;
        count++;
    }
    if (i <= 0 || i > count)
        return 0;
    LNode *b;
    b = (LNode *)malloc(sizeof(LNode));
    b->data = e;
    p = L;
    for (int j = 0; j < i - 1; j++)
        p = p->next;
    b->next = p->next;
    p->next = b;
    return 1;
}

// ListEmpty.h
status ListEmpty(LinkList L)
{
    if (L == NULL)
        return -1;
    else if (L->next == NULL)
        return 1;
    else
        return 0;
}
```

```
}
// ListDelete.h
status ListDelete(LinkList &L, int i, ElemType &e)
{
    if (L == NULL)
        return -1;
    LNode *p = L;
    LNode *q = L;
    int count = -1;
    while (p)
    {
        p = p->next;
        count++;
    }
    if (i <= 0 || i > count)
        return 0;
    p = L;
    for (int j = 0; j < i - 1; j++)
        p = p->next;
    e = p->next->data;
    q = p->next;
    p->next = q->next;
    free(q);
    return 1;
}
// GetElem.h
status GetElem(LinkList L, int i, ElemType &e)
{
    if (L == NULL)
        return -1;
    else
    {
        LinkList p = L->next;
        int count = 0;
        while (p != NULL)
        {
            p = p->next;
            count++;
        }
        if (i < 1 || i > count)
```

```
        return 0;
    else
    {
        p = L->next;
        for (int j = 0; j < i - 1; j++)
            p = p->next;
        e = p->data;
        return 1;
    }
}

// DestroyList.h
status DestroyList(LinkList &L)
{
    if (L == NULL)
        return -1;
    else
    {
        LinkList temp_1 = L->next;
        LinkList temp_2;
        while (temp_1 != NULL)
        {
            temp_2 = temp_1;
            temp_1 = temp_1->next;
            free(temp_2);
        }
        free(L);
        L = NULL;
        return 1;
    }
}

// ClearList.h
status ClearList(LinkList &L)
{
    if (L == NULL)
        return -1;
    else
    {
        LinkList temp_1 = L->next;
        LinkList temp_2;
```

```
        while (temp_1 != NULL)
        {
            temp_2 = temp_1;
            temp_1 = temp_1->next;
            free(temp_2);
        }
        L->next = NULL;
        return 1;
    }
}

// LocateList.h
int LocateList(LISTS Lists, char ListName[])
{
    for (int i = 0; i < Lists.length; i++)
    {
        if (strcmp(Lists.elem[i].name, ListName) == 0)
            return i + 1;
    }
    return 0;
}

// RemoveList.h
status RemoveList(LISTS &Lists, char ListName[])
{
    for (int i = 0; i < Lists.length; i++)
    {
        if (strcmp(Lists.elem[i].name, ListName) == 0)
        {
            LinkList temp_1 = Lists.elem[i].L->next;
            LinkList temp_2;
            while (temp_1 != NULL)
            {
                temp_2 = temp_1;
                temp_1 = temp_1->next;
                free(temp_2);
            }
            free(Lists.elem[i].L);
            Lists.elem[i].L = NULL;
            Lists.elem[i].name[0] = '\0';
            return 1;
        }
    }
}
```



```
    }
    return 0;
}
// LocateElem.h
status LocateElem(LinkList L, ElemType e)
{
    if (L == NULL)
        return -1;
    L = L->next;
    int count = 0;
    while (L)
    {
        count++;
        if (L->data == e)
            return count;
        L = L->next;
    }
    return 0;
}
// sortList.h
int sortList(LinkList L)
{
    if (L == NULL || L->next == NULL)
    {
        printf("线性表为空\n");
        return 0;
    }
    LinkList p = L->next;
    int count = 0;
    while (p != NULL)
    {
        p = p->next;
        count++;
    }
    int temp_array[count];
    p = L->next;
    for (int i = 0; i < count; i++, p = p->next)
    {
        temp_array[i] = p->data;
    }
}
```

```
    quickSort(temp_array, 0, count - 1);
    p = L->next;
    for (int i = 0; i < count; p = p->next, i++)
    {
        p->data = temp_array[i];
    }
    return 1;
}

void swap(int *a, int *b)
{
    int temp = *a;
    *a = *b;
    *b = temp;
}

int medianOfThree(int arr[], int low, int high)
{
    int mid = low + (high - low) / 2;
    if (arr[low] > arr[mid])
        swap(&arr[low], &arr[mid]);
    if (arr[low] > arr[high])
        swap(&arr[low], &arr[high]);
    if (arr[mid] > arr[high])
        swap(&arr[mid], &arr[high]);
    return mid;
}

int partition(int arr[], int low, int high)
{
    int pivotIndex = medianOfThree(arr, low, high);
    swap(&arr[pivotIndex], &arr[high]);
    int pivot = arr[high];
    int i = low - 1;
    for (int j = low; j < high; j++)
    {
        if (arr[j] < pivot)
        {
            i++;
            swap(&arr[i], &arr[j]);
        }
    }
    swap(&arr[i + 1], &arr[high]);
}
```

```
        return i + 1;
    }
void quickSort(int arr[], int low, int high)
{
    while (low < high)
    {
        int pi = partition(arr, low, high);
        if (pi - low < high - pi)
        {
            quickSort(arr, low, pi - 1);
            low = pi + 1;
        }
        else
        {
            quickSort(arr, pi + 1, high);
            high = pi - 1;
        }
    }
}
// reverseList.h
status reverseList(LinkList &L)
{
    if (L == NULL)
        return -1;
    if (L->next->next == NULL)
        return 0;
    LinkList pre = L->next->next, pro = L->next, temp = pro->next;
    pro->next = NULL;
    while (pre->next)
    {
        temp = pre;
        pre = pre->next;
        temp->next = pro;
        pro = temp;
    }
    temp = pre;
    temp->next = pro;
    pro = temp;
    L->next = pre;
    return 1;
}
```

```
}
// RemoveNthFromEnd.h
status RemoveNthFromEnd(LinkList &L, int n)
{
    if (L == NULL)
        return -1;
    LinkList p = L->next;
    int count = 0;
    while (p != NULL)
    {
        p = p->next;
        count++;
    }
    if (n > count || n < 0)
        return 0;
    p = L->next;
    for (int i = 1; i < count - n; i++)
    {
        p = p->next;
    }
    LinkList temp = p->next;
    p->next = temp->next;
    free(temp);
    return 1;
}
```

附录 C 基于二叉链表二叉树实现的源程序

```
#include "stdio.h"
#include "stdlib.h"
#include "string.h"
#include <windows.h>
#define TRUE 1
#define FALSE 0
#define OK 1
#define ERROR 0
#define INFEASIBLE -1
#define OVERFLOW -2
typedef int status;
typedef int KeyType;
typedef struct
{
    KeyType key;
    char others[20];
} TElemType; // 二叉树结点类型定义
typedef struct BiTNode
{ // 二叉链表结点的定义
    TElemType data;
    struct BiTNode *lchild, *rchild;
} BiTNode, *BiTree;
void visit(BiTree T)
{
    printf(" %d,%s", T->data.key, T->data.others);
}
typedef struct
{ // 线性表的管理表定义
    struct
    {
        char name[30];
        BiTree T;
    } elem[10];
    int length;
    int listsize;
} LIST;
int z, count_3;
void visit(BiTree T);
```

华中科技大学课程实验报告

```
BiTree create_Node(TElemType definition);
status SaveBiTree(BiTree T, char FileName[]);
status LoadBiTree(BiTree &T, char FileName[]);
status PreOrderTraverse(BiTree T);
status PostOrderTraverse(BiTree T);
struct BiTNode *LocateNode(BiTree T, KeyType e);
status LevelOrderTraverse(BiTree T);
status InsertNode(BiTree &T, KeyType e, int LR, TElemType c);
status InOrderTraverse(BiTree T);
struct BiTNode *GetSibling(BiTree T, KeyType e);
status DestroyBiTree(BiTree &T);
status DeleteNode(BiTree &T, KeyType e);
status ClearBiTree(BiTree &T);
status CreateBiTree_pre(BiTree &T, TElemType definition[]);
int BiTreeDepth(BiTree T);
status Assign(BiTree &T, KeyType e, TElemType value);
void AddList(LISTS &Lists, char ListName[]);
int LocateList(LISTS Lists, char ListName[]);
status RemoveList(LISTS &Lists, char ListName[]);
status InvertTree(BiTree &T);
void get_definition(TElemType definition[]);
void MaxPathSum(BiTree &T, int &result);
BiTree LowestCommonAncestor(BiTree &T, int p, int q);
BiTree create_tree(TElemType definition[]);
int main()
{
    SetConsoleOutputCP(CP_UTF8);
    LISTS Lists;
    Lists.length = 0;
    TElemType definition[100];
    int op = 1;
    int i;
    int temp, m;
    int result = 0;
    TElemType temp_2;
    int list_num = 1;
    int count = 0;
    char firstname[30];
    while (op)
    {
```

华中科技大学课程实验报告

```
char name_temp[30];
printf("\n\n");
printf("      Menu for non-Linear Table On non-Sequence Structur
e \n");
printf("-----\n");
if (count > 1)
{
    printf("创建的线性表有: \n");
    for (int temp_2 = 0; temp_2 < Lists.length; temp_2++)
    {
        if (strcmp(Lists.elem[temp_2].name, "0") != 0)
            printf("%s\n", Lists.elem[temp_2].name);
    }
    bool flag = 1;
    printf("请问你想对哪个线性表进行操作? \n");
    do
    {
        scanf("%s", name_temp);
        if (!(list_num = LocateList(Lists, name_temp)))
        {
            flag = 0;
            printf("找不到对应的线性表,请重新输入\n");
        }
        else
            flag = 1;
    } while (flag != OK);
}
printf("      1. CreateBiTree    12. PreOrderTraverse\n");
printf("      2. DestroyBiTree   13. InOrderTraverse\n");
printf("      3. ClearBiTree     14. PostOrderTraverse\n");
printf("      4. BiTreeEmpty     15. AddList\n");
printf("      5. BiTreeDepth     16. RemoveList\n");
printf("      6. Assign          17. LocateList\n");
printf("      7. LocateNode      18. LevelOrderTraverse\n");
printf("      8. GetSibling      19. InvertTree\n");
printf("      9. DeleteNode      20. MaxPathSum\n");
printf("     10. InsertNode      21. ShowAllList\n");
printf("     11. file_using      22. LowestCommonAncestor\n");
printf("      0. Exit\n");
printf("-----\n");
```

```

printf("    请选择你的操作[0~21]:");
scanf("%d", &op);
switch (op)
{
case 1: // 二叉树的创建
    if (Lists.length > 0)
    {
        printf("已初始化线性表\n");
        getchar();
        getchar();
        break;
    }
    printf("请输入你想创建的线性表名称\n");
    scanf("%s", firstname);
    if (strcmp(firstname, "0") == 0)
    {
        printf("名字非法\n");
        getchar();
        getchar();
        break;
    }
    AddList(Lists, firstname);
    printf("请问你想如何创建二叉树?\n关键字为0时,表示空子树\n-1表示\n\n");
    get_definition(definition);
    // if (i == 1)
    // {
    if (CreateBiTree_pre(Lists.elem[List.length - 1].T, defini
tion))
    {
        PreOrderTraverse(Lists.elem[List.length - 1].T);
        count++;
        printf("\n 创建成功\n");
    }
    else
        printf("创建失败, 有重复关键字\n");
    getchar();
    getchar();
    break;
case 2: // 销毁二叉树

```


华中科技大学课程实验报告

```
        if (DestroyBiTree(Lists.elem[list_num - 1].T) == OK)
            printf("销毁二叉树成功\n");
        else
            printf("二叉树已空\n");
            getchar();
            getchar();
            break;
    case 3: // 清空二叉树
        if (ClearBiTree(Lists.elem[list_num - 1].T->lchild) == OK &
            & ClearBiTree(Lists.elem[list_num - 1].T->rchild) == OK)
        {
            printf("清空二叉树成功\n");
        }
        else
            printf("二叉树已空\n");
            getchar();
            getchar();
            break;
    case 4: // 二叉树判空
        if (Lists.elem[list_num - 1].T == NULL)
            printf("二叉树为空\n");
        else if (Lists.elem[list_num - 1].T->lchild || Lists.elem[
list_num - 1].T->rchild || (Lists.elem[list_num - 1].T->lchild == NULL &
& Lists.elem[list_num - 1].T->rchild == NULL && Lists.elem[list_num - 1].T))
            printf("二叉树不为空\n");
        else
            printf("二叉树不存在\n");
            getchar();
            getchar();
            break;
    case 5: // 二叉树深度
        if ((i = BiTreeDepth(Lists.elem[list_num - 1].T)) > 0)
            printf("线性表深度为%d\n", i);
        else
            printf("线性表为空\n");
            getchar();
            getchar();
            break;
    case 6: // 节点赋值
        printf("您想更改哪个位置的值? \n");
```

华中科技大学课程实验报告

```
scanf("%d", &i);
printf("您想将其更改为什么值? \n");
scanf("%d %s", &temp_2.key, &temp_2.others);
if (Assign(Lists.elem[list_num - 1].T, i, temp_2))
{
    printf("赋值成功");
    PreOrderTraverse(Lists.elem[list_num - 1].T);
    printf("\n");
    InOrderTraverse(Lists.elem[list_num - 1].T);
    printf("\n");
}
else
    printf("赋值失败\n");
getchar();
getchar();
break;
case 7: // 查找结点
    printf("您想查找哪个元素? \n");
    scanf("%d", &i);
    BiTree p;
    if ((p = LocateNode(Lists.elem[list_num - 1].T, i)) != NULL)
    {
        printf("该节点为%d %s\n", p->data.key, p->data.others);
    }
    else
        printf("不存在该元素\n");
    getchar();
    getchar();
    break;
case 8: // 获得兄弟结点
    printf("您想查找哪个元素的兄弟? \n");
    scanf("%d", &i);
    if ((p = GetSibling(Lists.elem[list_num - 1].T, i)) != NULL)
        printf("该元素的兄弟结点
是%d %s\n", p->data.key, p->data.others);
    else
        printf("该元素无兄弟结点\n");
    getchar();
    getchar();
    break;
```

华中科技大学课程实验报告

```
case 9: // 删除结点
    printf("您想删除哪个位置的元素? \n");
    scanf("%d", &i);
    if (DeleteNode(Lists.elem[list_num - 1].T, i) == OK)
    {
        printf("该删除成功\n");
        PreOrderTraverse(Lists.elem[list_num - 1].T);
        printf("\n");
        InOrderTraverse(Lists.elem[list_num - 1].T);
        printf("\n");
    }
    else
        printf("删除操作失败\n");
    getchar();
    getchar();
    break;
case 10: // 插入元素
    printf("请输入要插入的元素\n");
    scanf("%d %s", &temp_2.key, &temp_2.others);
    printf("请输入要插入的位置\n");
    scanf("%d", &i);
    printf("0:左孩子\n1:右孩子\n-1:根节点\n");
    scanf("%d", &temp);
    if (InsertNode(Lists.elem[list_num - 1].T, i, temp, temp_2)
== OK)
    {
        printf("插入成功\n");
        PreOrderTraverse(Lists.elem[list_num - 1].T);
        printf("\n");
        InOrderTraverse(Lists.elem[list_num - 1].T);
        printf("\n");
    }
    else
        printf("插入失败\n");
    getchar();
    getchar();
    break;
case 11: // 文件读写
    char filename[30];
    printf("请输入指令\n");
```

华中科技大学课程实验报告

```
printf("1.文件读取\n");
printf("2.文件写入\n");
scanf("%d", &i);
if (i == 1)
{
    printf("请输入需要读取的文件地址,请用.dat 作为后缀\n");
    scanf("%s", filename);
    printf("请输入你想创建的线性表名称\n");
    scanf("%s", firstname);
    if ((temp = LocateList(Lists, firstname)) > 0)
    {
        printf("有重复名称,无法创建\n");
        break;
    }
    AddList(Lists, firstname);
    if (LoadBiTree(Lists.elem[List.length - 1].T, filename
) == OK)
    {
        printf("读取成功\n");
        count++;
    }
    else
        printf("读取失败\n");
}
else if (i == 2)
{
    printf("请输入需要写入的文件地址,请用.dat 作为后缀\n");
    scanf("%s", filename);
    if (SaveBiTree(Lists.elem[list_num - 1].T, filename) ==
OK)
        printf("写入成功\n");
    else
        printf("写入失败\n");
}
getchar();
getchar();
break;
case 12: // 前序遍历
    PreOrderTraverse(Lists.elem[list_num - 1].T);
    printf("\n");
```

华中科技大学课程实验报告

```
        getchar();
        getchar();
        break;
    case 13: // 中序遍历
        InOrderTraverse(Lists.elem[list_num - 1].T);
        printf("\n");
        getchar();
        getchar();
        break;
    case 15: // 多线性表管理: 增加一个线性表
        printf("请输入新线性表名字\n");
        scanf("%s", firstname);
        if ((temp = LocateList(Lists, firstname)) > 0)
        {
            printf("有重复名称,无法创建\n");
            break;
        }
        AddList(Lists, firstname);
        printf("请问你想如何创建二叉树?\n 关键字为 0 时,表示空子树,-1 表示\n\n输入结束\n");
        get_definition(definition);
        if (CreateBiTree_pre(Lists.elem[Lists.length - 1].T, definition))
        {
            PreOrderTraverse(Lists.elem[Lists.length - 1].T);
            count++;
            printf("\n 创建成功\n");
        }
        else
            printf("创建失败, 有重复关键字\n");
        getchar();
        getchar();
        break;
    case 16: // 多线性表管理: 移除线性表
        printf("请输入需要移除的线性表名字\n");
        scanf("%s", firstname);
        if (RemoveList(Lists, firstname) == OK)
            printf("移除成功\n");
        else
            printf("移除失败\n");
```

华中科技大学课程实验报告

```
        getchar();
        getchar();
        break;
case 17: // 多线性表管理: 查找线性表
    printf("请输入需要查找的线性表名字\n");
    scanf("%s", firstname);
    if ((temp = LocateList(Lists, firstname)) == 0)
        printf("查找失败\n");
    else
        printf("查找成功,位置为%d\n", temp);
    getchar();
    getchar();
    break;
case 14: // 后序遍历
    PostOrderTraverse(Lists.elem[list_num - 1].T);
    printf("\n");
    getchar();
    getchar();
    break;
case 19: // 二叉树翻转
    if (InvertTree(Lists.elem[list_num - 1].T))
    {
        printf("反转成功,反转后的线性表为: \n");
        PreOrderTraverse(Lists.elem[list_num - 1].T);
        printf("\n");
        InOrderTraverse(Lists.elem[list_num - 1].T);
        printf("\n");
    }
    else
        printf("翻转失败");
    getchar();
    getchar();
    break;
case 18: // 层序遍历
    LevelOrderTraverse(Lists.elem[list_num - 1].T);
    printf("\n");
    getchar();
    getchar();
    break;
case 20: // 最大路径和
```

华中科技大学课程实验报告

```
        result = 0;
        MaxPathSum(Lists.elem[list_num - 1].T, result);
        printf("最大路径和为%d\n", result);
        getchar();
        getchar();
        break;
    case 21: // show all lists
        printf("创建的线性表有: \n");
        for (int temp_2 = 0; temp_2 < Lists.length; temp_2++)
        {
            if (strcmp(Lists.elem[temp_2].name, "0") != 0)
                printf("%s\n", Lists.elem[temp_2].name);
        }
        getchar();
        getchar();
        break;
    case 22: // 最近公共祖先
        printf("请输入要求的两个节点数据\n");
        scanf("%d %d", &temp, &i);
        BiTree t;
        t = LowestCommonAncestor(Lists.elem[list_num - 1].T, temp, i);
        if (t)
            printf("最近共同祖先是:%d", t->data.key);
        else
            printf("无法找到");
        getchar();
        getchar();
        break;
    case 0:
        break;
    } // end of switch
} // end of while
printf("欢迎下次再使用本系统! \n");
return 0;
} // end of main()

BiTree create_Node(TElemType definition)
{
    BiTree Node = (BiTree)malloc(sizeof(struct BiTNode));
    Node->data.key = definition.key;
    strcpy(Node->data.others, definition.others);
}
```

华中科技大学课程实验报告

```
Node->lchild = Node->rchild = NULL;
return Node;
}
// file_using.h
status SaveBiTree(BiTree T, char FileName[])
{
    int i = 0;
    if (!T)
        return -1;
    FILE *fp = fopen(FileName, "wb");
    BiTree stack[100];
    TElemType definition[100];
    BiTree p = T;
    int top = 0;
    if (T)
        stack[top++] = p;
    while (top)
    {
        p = stack[--top];
        if (p)
        {
            definition[i].key = p->data.key;
            strcpy(definition[i++].others, p->data.others);
        }
        else
        {
            definition[i].key = 0;
            strcpy(definition[i++].others, "null");
        }
        if (p)
        {
            stack[top++] = p->rchild;
        }
        if (p)
            stack[top++] = p->lchild;
    }
    fwrite(&i, sizeof(int), 1, fp);
    fwrite(&definition, sizeof(TElemType), i, fp);
    fclose(fp);
    return 1;
}
```


华中科技大学课程实验报告

```
}
status LoadBiTree(BiTree &T, char FileName[])
// 读入文件FileName的结点数据, 创建二叉树
{
    // 请在这里补充代码, 完成本关任务
    /***** Begin 2 *****/
    z = 0;
    FILE *fp = fopen(FileName, "rb");
    fread(&count_3, sizeof(int), 1, fp);
    TElemType definition[count_3];
    fread(&definition, sizeof(TElemType), count_3, fp);
    T = create_tree(definition);
    return 1;
    /***** End 2 *****/
}

BiTree create_tree(TElemType definition[])
{
    if (definition[z].key == 0)
    {
        z++;
        return NULL;
    }
    BiTree root = create_Node(definition[z]);
    if (z == count_3)
        return root;
    z++;
    root->lchild = create_tree(definition);
    root->rchild = create_tree(definition);
    return root;
}

// PreOrderTraverse.h
status PreOrderTraverse(BiTree T)
{
    if (T)
    {
        visit(T);
        PreOrderTraverse(T->lchild);
        PreOrderTraverse(T->rchild);
    }
    return 1;
}
```

```
}
// PostOrderTraverse.h
status PostOrderTraverse(BiTree T)
{
    if (T)
    {
        PostOrderTraverse(T->lchild);
        PostOrderTraverse(T->rchild);
        visit(T);
    }
    return 1;
}
// LocateNode.h
struct BiTNode *LocateNode(BiTree T, KeyType e)
{
    struct BiTNode *stack[100];
    struct BiTNode *p = T;
    int top = 0;
    if (T)
        stack[top++] = T;
    while (top)
    {
        p = stack[--top];
        if (p->data.key == e)
            return p;
        if (p->rchild != NULL)
            stack[top++] = p->rchild;
        if (p->lchild != NULL)
            stack[top++] = p->lchild;
    }
    return NULL;
}
// LevelOrderTraverse.h
status LevelOrderTraverse(BiTree T)
{
    BiTree queue[100];
    int front = 0, rear = 0;
    if (T)
        queue[rear++] = T;
    BiTree p = T;
```

```
while (front != rear)
{
    if (p->lchild)
        queue[rear++] = p->lchild;
    if (p->rchild)
        queue[rear++] = p->rchild;
    visit(p);
    p = queue[++front];
}
return 1;
}
// InsertNode.h
int list_treaverse(BiTree T, TElemType value)
{
    struct BiTNode *stack[100];
    BiTree p = T;
    int top = 0;
    if (T)
        stack[top++] = p;
    while (top)
    {
        p = stack[--top];
        if (p->data.key == value.key)
            return 1;
        if (p->rchild)
            stack[top++] = p->rchild;
        if (p->lchild)
            stack[top++] = p->lchild;
    }
    return 0;
}
status InsertNode(BiTree &T, KeyType e, int LR, TElemType c)
{
    if (list_treaverse(T, c))
        return 0;
    BiTree p;
    p = (BiTree)malloc(sizeof(struct BiTNode));
    p->data = c;
    if (LR == -1)
    {
```

华中科技大学课程实验报告

```
p->rchild = T;
p->lchild = NULL;
T = p;
return 1;
}
struct BiTNode *stack[100];
BiTree q = T;
int top = 0;
if (T)
    stack[top++] = q;
while (top)
{
    q = stack[--top];
    if (q->data.key == e)
    {
        if (LR == 0)
        {
            BiTree temp = q->lchild;
            q->lchild = p;
            p->rchild = temp;
            p->lchild = NULL;
        }
        else if (LR == 1)
        {
            BiTree temp = q->rchild;
            q->rchild = p;
            p->rchild = temp;
            p->lchild = NULL;
        }
        return 1;
    }
    if (q->rchild)
        stack[top++] = q->rchild;
    if (q->lchild)
        stack[top++] = q->lchild;
}
return 0;
}
// InOrderTraverse.h
status InOrderTraverse(BiTree T)
```

华中科技大学课程实验报告

```
{
    if (T)
    {
        InOrderTraverse(T->lchild);
        visit(T);
        InOrderTraverse(T->rchild);
    }
    return 1;
}

// GetSibling.h
struct BiTNode *GetSibling(BiTree T, KeyType e)
{
    struct BiTNode *stack[100];
    BiTree p = T;
    int top = 0;
    if (T)
        stack[top++] = p;
    while (top)
    {
        p = stack[--top];
        if (p->rchild != NULL && p->lchild != NULL && p->lchild->data.key == e)
            return p->rchild;
        if (p->lchild != NULL && p->rchild != NULL && p->rchild->data.key == e)
            return p->lchild;
        if (p->rchild)
            stack[top++] = p->rchild;
        if (p->lchild)
            stack[top++] = p->lchild;
    }
    return NULL;
}

// DestroyBiTree.h
status DestroyBiTree(BiTree &T)
{
    if (T)
    {
        DestroyBiTree(T->lchild);
        DestroyBiTree(T->rchild);
    }
}
```

华中科技大学课程实验报告

```
        free(T);
        T = NULL;
    }
    return 1;
}
// DeleteNode.h
void delete_right_node(BiTree &q, BiTree &p, KeyType e)
{
    BiTree temp = p;
    if (p->lchild == NULL && p->rchild == NULL)
    {
        q->rchild = NULL;
        free(p);
    }
    else if (p->lchild == NULL && p->rchild != NULL)
    {
        q->rchild = p->rchild;
        free(p);
    }
    else if (p->rchild == NULL && p->lchild != NULL)
    {
        q->rchild = p->lchild;
        free(p);
    }
    else if (p->lchild != NULL && p->rchild != NULL)
    {
        q->rchild = p->lchild;
        BiTree temp2 = p->rchild;
        BiTree t = p->lchild;
        free(p);
        while (t->rchild)
            t = t->rchild;
        t->rchild = temp2;
    }
}
void delete_left_node(BiTree &q, BiTree &p, KeyType e)
{
    BiTree temp = p;
    if (p->lchild == NULL && p->rchild == NULL)
    {
```

```
        q->lchild = NULL;
        free(p);
    }
    else if (p->lchild == NULL && p->rchild != NULL)
    {
        q->lchild = p->rchild;
        free(p);
    }
    else if (p->rchild == NULL && p->lchild != NULL)
    {
        q->lchild = p->lchild;
        free(p);
    }
    else if (p->lchild != NULL && p->rchild != NULL)
    {
        q->lchild = p->lchild;
        BiTree temp2 = p->rchild;
        BiTree t = p->lchild;
        free(p);
        while (t->rchild)
            t = t->rchild;
        t->rchild = temp2;
    }
}

status DeleteNode(BiTree &T, KeyType e)
{
    struct BiTNode *stack[100];
    BiTree p = T;
    if (p->data.key == e)
    {
        if (p->lchild == NULL && p->rchild == NULL)
            free(p);
        else if (p->lchild == NULL && p->rchild != NULL)
        {
            T = p->rchild;
            free(p);
        }
        else if (p->rchild == NULL && p->lchild != NULL)
        {
            T = p->lchild;
```

```
        free(p);
    }
    else if (p->lchild != NULL && p->rchild != NULL)
    {
        T = T->lchild;
        BiTree temp = p->rchild;
        struct BiTNode *temp_2 = p->lchild;
        free(p);
        while (temp_2->rchild)
            temp_2 = temp_2->rchild;
        temp_2->rchild = temp;
    }
    return 1;
}
int top = 0;
if (T)
    stack[top++] = p;
while (top)
{
    p = stack[--top];
    if (p->lchild != NULL && p->lchild->data.key == e)
    {
        delete_left_node(p, p->lchild, e);
        return 1;
    }
    if (p->rchild != NULL && p->rchild->data.key == e)
    {
        delete_right_node(p, p->rchild, e);
        return 1;
    }
    if (p->rchild)
        stack[top++] = p->rchild;
    if (p->lchild)
        stack[top++] = p->lchild;
}
return 0;
}
// ClearBiTree.h
status ClearBiTree(BiTree &T)
{
```



```
    if (T && (T->lchild || T->rchild))
    {
        ClearBiTree(T->lchild);
        ClearBiTree(T->rchild);
        free(T);
        T = NULL;
    }
    return 1;
}

// CreateBiTree_pre.h
int count_1 = 0;
BiTree create_tree_pre(TElemType definition[])
{
    if (definition[count_1].key == 0)
    {
        count_1++;
        return NULL;
    }
    BiTree root = create_Node(definition[count_1]);
    if (definition[count_1].key == -1)
        return root;
    count_1++;
    root->lchild = create_tree_pre(definition);
    root->rchild = create_tree_pre(definition);
    return root;
}

status CreateBiTree_pre(BiTree &T, TElemType definition[])
{
    count_1 = 0;
    int temp[200000] = {0};
    for (int i = 0; definition[i].key != -1; i++)
    {
        temp[definition[i].key]++;
        if (temp[definition[i].key] > 1 && definition[i].key != 0)
        {
            return 0;
        }
    }
    T = create_tree_pre(definition);
    return 1;
}
```

华中科技大学课程实验报告

```
}
// BiTreeDepth.h
int BiTreeDepth(BiTree T)
{
    int leftdepth = 0, rightdepth = 0;
    if (T && T->lchild == NULL && T->rchild == NULL)
        return 1;
    else if (!T)
        return 0;
    else if (T->lchild || T->rchild)
    {
        if (T->lchild)
            leftdepth = BiTreeDepth(T->lchild);
        if (T->rchild)
            rightdepth = BiTreeDepth(T->rchild);
    }
    int result = 1 + (leftdepth > rightdepth ? leftdepth : rightdepth);
    return result;
}
// Assign.h
int list_treaverse(BiTree T, KeyType e, TElemType value)
{
    struct BiTNode *stack[100];
    BiTree p = T;
    int top = 0;
    if (T)
        stack[top++] = p;
    while (top)
    {
        p = stack[--top];
        if (p->data.key == value.key && p->data.key != e)
            return 1;
        if (p->rchild)
            stack[top++] = p->rchild;
        if (p->lchild)
            stack[top++] = p->lchild;
    }
    return 0;
}
status Assign(BiTree &T, KeyType e, TElemType value)
```

```
{
    struct BiTNode *stack[100];
    BiTree p = T;
    int top = 0;
    if (list_treaverse(T, e, value))
        return 0;
    if (T)
        stack[top++] = p;
    while (top)
    {
        p = stack[--top];
        if (p->data.key == e)
        {
            p->data.key = value.key;
            strcpy(p->data.others, value.others);
            return 1;
        }
        if (p->rchild)
            stack[top++] = p->rchild;
        if (p->lchild)
            stack[top++] = p->lchild;
    }
    return 0;
}

// AddList.h
void AddList(LISTS &Lists, char ListName[])
{
    Lists.elem[Lists.length++].T = (BiTree)malloc(sizeof(struct BiTNode));
    Lists.elem[Lists.length - 1].T->lchild = NULL;
    Lists.elem[Lists.length - 1].T->rchild = NULL;
    strcpy(Lists.elem[Lists.length - 1].name, ListName);
}

// LocateList.h
int LocateList(LISTS Lists, char ListName[])
{
    for (int i = 0; i < Lists.length; i++)
    {
        if (strcmp(Lists.elem[i].name, ListName) == 0)
            return i + 1;
    }
}
```

```
    }
    return 0;
}
// RemoveList.h
status RemoveList(LISTS &Lists, char ListName[])
{
    for (int i = 0; i < Lists.length; i++)
    {
        if (strcmp(Lists.elem[i].name, ListName) == 0)
        {
            DestroyBiTree(Lists.elem[i].T);
            strcpy(Lists.elem[i].name, "0");
            return 1;
        }
    }
    return 0;
}
// reverseTree.h
status InvertTree(BiTree &T)
{
    if (T)
    {
        BiTree p = T->lchild;
        T->lchild = T->rchild;
        T->rchild = p;
        InvertTree(T->lchild);
        InvertTree(T->rchild);
    }
    return 1;
}
// get_definition.h
void get_definition(TElemType definition[])
{
    int i = 0;
    do
    {
        scanf("%d%s", &definition[i].key, definition[i].others);
    } while (definition[i++].key != -1);
    return;
}
```

华中科技大学课程实验报告

```
// MaxPathSum.h
void travers(BiTree &p, int &result, int &temp_result)
{
    temp_result = temp_result + p->data.key;
    if (p->lchild == NULL && p->rchild == NULL)
    {
        result = temp_result > result ? temp_result : result;
    }
    if (p->lchild)
    {
        travers(p->lchild, result, temp_result);
        temp_result = temp_result - p->lchild->data.key;
    }
    if (p->rchild)
    {
        travers(p->rchild, result, temp_result);
        temp_result = temp_result - p->rchild->data.key;
    }
    return;
}

void MaxPathSum(BiTree &T, int &result)
{
    BiTree p = T;
    int temp_result = 0;
    travers(p, result, temp_result);
    return;
}

// LowestCommonAncestor.h
BiTree LowestCommonAncestor(BiTree &T, int p, int q)
{
    if (T == NULL)
        return NULL;
    if (T->data.key == p || T->data.key == q)
        return T;
    BiTree l = LowestCommonAncestor(T->lchild, p, q);
    BiTree r = LowestCommonAncestor(T->rchild, p, q);
    if (l && r)
        return T;
    else if (l && !r)
        return l;
}
```

华中科技大学课程实验报告

```
else if (!l && r)
    return r;
else
    return NULL;
}
```

附录 D 基于邻接表图实现的源程序

```
#include <stdio.h>
#include <stdlib.h>
#include <string.h>
#include <windows.h>
// def.h
#define MAX_VERTEX_NUM 30
#define OK 1
#define ERROR 0
#define TRUE 1
#define FALSE 0
#define INFEASIBLE -1
#define OVERFLOW -2
typedef int status;
typedef int KeyType;
typedef struct
{
    KeyType key;
    char others[20];
} VertexType;
typedef struct ArcNode
{
    int adjvex;
    struct ArcNode *nextarc;
} ArcNode;
typedef struct VNode
{
    VertexType data;
    ArcNode *firstarc;
} VNode, AdjList[MAX_VERTEX_NUM];
typedef struct
{
    AdjList vertices;
    int vexnum, arcnum;
    int kind;
} ALGraph;
typedef struct
{
    struct
```

```
    char name[30];
    ALGraph T;
} elem[10];
int length;
int listsize;
} LISTS;
int CreateGraph(ALGraph &G, VertexType V[], KeyType VR[][2]);
int DestroyGraph(ALGraph &G);
int LocateVex(ALGraph G, KeyType u);
int PutVex(ALGraph &G, KeyType u, VertexType value);
int FirstAdjVex(ALGraph G, KeyType u);
int NextAdjVex(ALGraph G, KeyType v, KeyType w);
int InsertVex(ALGraph &G, VertexType v);
int DeleteVex(ALGraph &G, KeyType v);
int InsertArc(ALGraph &G, KeyType v, KeyType w);
int DeleteArc(ALGraph &G, KeyType v, KeyType w);
void DFS(ALGraph G, int v, int *visited);
int DFSTraverse(ALGraph G);
int BFSTraverse(ALGraph G);
void dfs_k(ALGraph G, int v, int k, int depth, int *visited);
int VerticesSetLessThanK(ALGraph G, KeyType v, int k);
int AddList(LISTS &Lists, char ListName[]);
int LocateList(LISTS Lists, char ListName[]);
int RemoveList(LISTS &Lists, char ListName[]);
int ShortestPathLength(ALGraph G, KeyType u, KeyType v);
void dfs_conn(ALGraph G, int v, int *visited);
int ConnectedComponentsNums(ALGraph G);
int SaveGraph(ALGraph G, char FileName[]);
int LoadGraph(ALGraph &G, char FileName[]);
int main()
{
    SetConsoleOutputCP(CP_UTF8);
    LISTS Lists;
    Lists.length = 0;
    int op = 1;
    int i;
    int temp;
    int temp_2;
    int list_num = 1;
    int count = 0;
```


华中科技大学课程实验报告

```
char firstname[30];
while (op)
{
    char name_temp[30];
    printf("\n\n");
    printf("      Menu for non-Linear Table On Graph Structure \n");
    printf("-----\n");
    if (count > 1)
    {
        printf("创建的线性表有: \n");
        for (int temp_2 = 0; temp_2 < Lists.length; temp_2++)
        {
            if (strcmp(Lists.elem[temp_2].name, "0") != 0)
                printf("%s\n", Lists.elem[temp_2].name);
        }
        bool flag = 1;
        printf("请问你想对哪个线性表进行操作? \n");
        do
        {
            scanf("%s", name_temp);
            if (!(list_num = LocateList(Lists, name_temp)))
            {
                flag = 0;
                printf("找不到对应的线性表,请重新输入\n");
            }
            else
                flag = 1;
        } while (flag != OK);
    }
    printf("      1. CreateGraph   11. file_using\n");
    printf("      2. DestroyGraph  12. DFSTraverse\n");
    printf("      3. LocateVex     13. BFSTraverse\n");
    printf("      4. PutVex        14. VerticesSetLessThanK\n");
    printf("      5. FirstAdjVex   15. AddList\n");
    printf("      6. NextAdjVex    16. RemoveList\n");
    printf("      7. InsertVex     17. LocateList\n");
    printf("      8. DeleteVex     18. ShortestPathLength\n");
    printf("      9. InsertArc     19. ConnectedComponentsNums\n");
);
    printf("      10. DeleteArc    20. ShowAllList\n\n");
```

华中科技大学课程实验报告

```
printf("          0. Exit\n");
printf("-----\n");
printf("    请选择你的操作[0~20]:");
scanf("%d", &op);
switch (op)
{
case 1: // 图的创建
    if (Lists.length > 0)
    {
        printf("已初始化线性表\n");
        getchar();
        getchar();
        break;
    }
    printf("请输入你想创建的线性表名称\n");
    scanf("%s", firstname);
    if (strcmp(firstname, "0") == 0)
    {
        printf("名字非法\n");
        getchar();
        getchar();
        break;
    }
    AddList(Lists, firstname);
    VertexType V[30];
    KeyType VR[100][2];
    i = 0;
    int j;
    printf("请输入节点信息\n");
    do
    {
        scanf("%d%s", &V[i].key, V[i].others);
    } while (V[i++].key != -1);
    printf("请输入边\n");
    i = 0;
    do
    {
        scanf("%d%d", &VR[i][0], &VR[i][1]);
    } while (VR[i++][0] != -1);
    if (CreateGraph(Lists.elem[List.length - 1].T, V, VR))
```

华中科技大学课程实验报告

```
{
    printf("创建成功\n");
    for (j = 0; j < Lists.elem[Lists.length - 1].T.vexnum; j++)
    {
        ArcNode *p = Lists.elem[Lists.length - 1].T.vertices[j].firstarc;
        printf("%d %s", Lists.elem[Lists.length - 1].T.vertices[j].data.key, Lists.elem[Lists.length - 1].T.vertices[j].data.other_s);

        while (p)
        {
            printf(" %d", p->adjvex);
            p = p->nextarc;
        }
        printf("\n");
    }
    count++;
}
else
    printf("创建失败，有重复关键字\n");
getchar();
getchar();
break;
case 2: // 销毁图
    if (DestroyGraph(Lists.elem[list_num - 1].T) == OK)
        printf("销毁图成功\n");
    getchar();
    getchar();
    break;
case 3: // 查找顶点
    printf("请输入想要查找的顶点\n");
    scanf("%d", &i);
    if (!LocateVex(Lists.elem[list_num - 1].T, i))
        printf("查找失败\n");
    getchar();
    getchar();
    break;
case 4: // 顶点赋值
    printf("请输入你想赋值的顶点\n");
    scanf("%d", &i);
```

华中科技大学课程实验报告

```
printf("请输入新值\n");
VertexType value;
scanf("%d%s", &value.key, value.others);
if (PutVex(Lists.elem[list_num - 1].T, i, value))
{
    printf("赋值成功\n");
    for (j = 0; j < Lists.elem[List.length - 1].T.vexnum; j++)
    {
        ArcNode *p = Lists.elem[List.length - 1].T.vertices[j].firstarc;
        printf("%d %s", Lists.elem[List.length - 1].T.vertices[j].data.key, Lists.elem[List.length - 1].T.vertices[j].data.others);

        while (p)
        {
            printf(" %d", p->adjvex);
            p = p->nextarc;
        }
        printf("\n");
    }
    getchar();
    getchar();
    break;
case 5: // 获得第一邻接点
    printf("请输入你想求的节点\n");
    scanf("%d", &i);
    if (FirstAdjVex(Lists.elem[List.length - 1].T, i) == -1)
        printf("查找失败");
    getchar();
    getchar();
    break;
case 6: // 获得下一邻接点
    printf("请输入你想求的节点\n");
    scanf("%d", &i);
    printf("请输入该节点的邻接节点\n");
    scanf("%d", &temp_2);
    if (NextAdjVex(Lists.elem[List.length - 1].T, i, temp_2) =
= -1)
        printf("查找失败");
```

华中科技大学课程实验报告

```
        getchar();
        getchar();
        break;
    case 7: // 插入顶点
        printf("请输入插入顶点信息\n");
        scanf("%d%s", &value.key, value.others);
        if (InsertVex(Lists.elem[Lists.length - 1].T, value))
        {
            printf("插入成功\n");
            for (j = 0; j < Lists.elem[Lists.length - 1].T.vexnum; j++)
            {
                ArcNode *p = Lists.elem[Lists.length - 1].T.vertices[j].firstarc;
                printf("%d %s", Lists.elem[Lists.length - 1].T.vertices[j].data.key, Lists.elem[Lists.length - 1].T.vertices[j].data.others);
                while (p)
                {
                    printf(" %d", p->adjvex);
                    p = p->nextarc;
                }
                printf("\n");
            }
        }
        else
        {
            printf("操作失败\n");
            getchar();
            getchar();
            break;
        }
    case 8: // 删除顶点
        printf("请输入你想删除的节点\n");
        scanf("%d", &i);
        if (DeleteVex(Lists.elem[Lists.length - 1].T, i))
        {
            printf("删除成功\n");
            for (j = 0; j < Lists.elem[Lists.length - 1].T.vexnum; j++)
            {
                ArcNode *p = Lists.elem[Lists.length - 1].T.vertices[j].firstarc;
```

华中科技大学课程实验报告

```
        printf("%d %s", Lists.elem[Lists.length - 1].T.vertices[j].data.key, Lists.elem[Lists.length - 1].T.vertices[j].data.other
s);

        while (p)
        {
            printf(" %d", p->adjvex);
            p = p->nextarc;
        }
        printf("\n");
    }
}
else
    printf("操作失败\n");
getchar();
getchar();
break;
case 9: // 插入弧
    printf("请输入想插入的弧\n");
    scanf("%d %d", &i, &temp_2);
    if (InsertArc(Lists.elem[Lists.length - 1].T, i, temp_2))
    {
        printf("插入成功\n");
        for (j = 0; j < Lists.elem[Lists.length - 1].T.vexnum; j++)
        {
            ArcNode *p = Lists.elem[Lists.length - 1].T.vertices[j].firstarc;
            printf("%d %s", Lists.elem[Lists.length - 1].T.vertices[j].data.key, Lists.elem[Lists.length - 1].T.vertices[j].data.other
s);

            while (p)
            {
                printf(" %d", p->adjvex);
                p = p->nextarc;
            }
            printf("\n");
        }
    }
else
    printf("操作失败\n");
getchar();
```

华中科技大学课程实验报告

```
        getchar();
        break;
    case 10: // 删除弧
        printf("请输入想删除的弧\n");
        scanf("%d %d", &i, &temp_2);
        if (DeleteArc(Lists.elem[Lists.length - 1].T, i, temp_2))
        {
            printf("删除成功\n");
            for (j = 0; j < Lists.elem[Lists.length - 1].T.vexnum; j++)
            {
                ArcNode *p = Lists.elem[Lists.length - 1].T.vertices[j].firstarc;
                printf("%d %s", Lists.elem[Lists.length - 1].T.vertices[j].data.key, Lists.elem[Lists.length - 1].T.vertices[j].data.other_s);

                while (p)
                {
                    printf(" %d", p->adjvex);
                    p = p->nextarc;
                }
                printf("\n");
            }
        }
        else
            printf("操作失败\n");
        getchar();
        getchar();
        break;
    case 11: // 文件读写
        char filename[30];
        printf("请输入指令\n");
        printf("1.文件读取\n");
        printf("2.文件写入\n");
        scanf("%d", &i);
        if (i == 1)
        {
            printf("请输入需要读取的文件地址,请用.dat 作为后缀\n");
            scanf("%s", filename);
            printf("请输入你想创建的图名称\n");
            scanf("%s", firstname);
```

华中科技大学课程实验报告

```
        if ((temp = LocateList(Lists, firstname)) > 0)
        {
            printf("有重复名称,无法创建\n");
            break;
        }
        AddList(Lists, firstname);
        if (LoadGraph(Lists.elem[List.length - 1].T, filename)
== OK)
        {
            printf("读取成功\n");
            for (j = 0; j < Lists.elem[List.length - 1].T.vexnum; j++)
            {
                ArcNode *p = Lists.elem[List.length - 1].T.vertices[j].firstarc;
                printf("%d %s", Lists.elem[List.length - 1].T.vertices[j].data.key, Lists.elem[List.length - 1].T.vertices[j].data.others);

                while (p)
                {
                    printf(" %d", p->adjvex);
                    p = p->nextarc;
                }
                printf("\n");
            }
            count++;
        }
        else
            printf("读取失败\n");
    }
    else if (i == 2)
    {
        printf("请输入需要写入的文件地址,请用.dat 作为后缀\n");
        scanf("%s", filename);
        if (SaveGraph(Lists.elem[list_num - 1].T, filename) == OK)
            printf("写入成功\n");
        else
            printf("写入失败\n");
    }
    getchar();
```


华中科技大学课程实验报告

```
        getchar();
        break;
case 12: // 深度优先搜索遍历
    DFSTraverse(Lists.elem[list_num - 1].T);
    printf("\n");
    getchar();
    getchar();
    break;
case 13: // 广度优先搜索遍历
    BFSTraverse(Lists.elem[list_num - 1].T);
    printf("\n");
    getchar();
    getchar();
    break;
case 15: // 多线性表管理: 增加一个线性表
    printf("请输入新线性表名字\n");
    scanf("%s", firstname);
    if ((temp = LocateList(Lists, firstname)) > 0)
    {
        printf("有重复名称,无法创建\n");
        break;
    }
    AddList(Lists, firstname);
    printf("请输入节点信息\n");
    do
    {
        scanf("%d%s", &V[i].key, V[i].others);
    } while (V[i++].key != -1);
    printf("请输入边\n");
    i = 0;
    do
    {
        scanf("%d%d", &VR[i][0], &VR[i][1]);
    } while (VR[i++][0] != -1);
    if (CreateGraph(Lists.elem[List.length - 1].T, V, VR))
    {
        for (j = 0; j < Lists.elem[List.length - 1].T.vexnum; j++)
        {
            ArcNode *p = Lists.elem[List.length - 1].T.vertexes[j].firstarc;
```

华中科技大学课程实验报告

```
        printf("%d %s", Lists.elem[Lists.length - 1].T.vertices[j].data.key, Lists.elem[Lists.length - 1].T.vertices[j].data.other
s);

        while (p)
        {
            printf(" %d", p->adjvex);
            p = p->nextarc;
        }
        printf("\n");
    }
    count++;
    printf("\n 创建成功\n");
}
else
    printf("创建失败, 有重复关键字\n");
getchar();
getchar();
break;

case 16: // 多线性表管理: 移除线性表
    printf("请输入需要移除的线性表名字\n");
    scanf("%s", firstname);
    if (RemoveList(Lists, firstname) == OK)
        printf("移除成功\n");
    else
        printf("移除失败\n");
    getchar();
    getchar();
    break;

case 17: // 多线性表管理: 查找线性表
    printf("请输入需要查找的线性表名字\n");
    scanf("%s", firstname);
    if ((temp = LocateList(Lists, firstname)) == 0)
        printf("查找失败\n");
    else
        printf("查找成功, 位置为%d\n", temp);
    getchar();
    getchar();
    break;

case 14: // 距离小于 k 的顶点集合
    printf("请输入顶点\n");
```

华中科技大学课程实验报告

```
scanf("%d", &i);
printf("请输入距离 k\n");
scanf("%d", &temp_2);
VerticesSetLessThanK(Lists.elem[List.length - 1].T, i, temp_2);

getchar();
getchar();
break;

case 18: // 顶点间最短路径和长度
printf("请输入两个顶点的信息\n");
scanf("%d %d", &i, &temp_2);
i = ShortestPathLength(Lists.elem[List.length - 1].T, i, temp_2);

if (i != 1000 && i != -2)
    printf("最短路径为:%d", i);
else if (i == -2)
    printf("无效节点\n");
else
    printf("两点之间没有通路\n");
getchar();
getchar();
break;

case 19: // 图的连通分量
i = ConnectedComponentsNums(Lists.elem[List.length - 1].T);
printf("连通分量个数为:%d", i);
getchar();
getchar();
break;

case 20: // show all lists
printf("创建的线性表有: \n");
for (int temp_2 = 0; temp_2 < Lists.length; temp_2++)
{
    if (strcmp(Lists.elem[temp_2].name, "0") != 0)
        printf("%s\n", Lists.elem[temp_2].name);
}
getchar();
getchar();
break;

case 0:
break;
```

华中科技大学课程实验报告

```
        } // end of switch
    } // end of while
    printf("欢迎下次再使用本系统! \n");
    return 0;
} // end of main()
// CreateGraph.h
int CreateGraph(ALGraph &G, VertexType V[], KeyType VR[][2])
{
    int i, j, k;
    G.vexnum = 0;
    G.arcnum = 0;
    // 录入顶点
    for (i = 0; V[i].key != -1; i++)
    {
        for (j = 0; j < G.vexnum; j++)
        {
            if (G.vertices[j].data.key == V[i].key)
                return 0; // 重复关键字
        }
        G.vertices[G.vexnum].data = V[i];
        G.vertices[G.vexnum].firstarc = NULL;
        G.vexnum++;
    }
    // 录入边
    for (i = 0; VR[i][0] != -1; i++)
    {
        int v = -1, w = -1;
        for (j = 0; j < G.vexnum; j++)
        {
            if (G.vertices[j].data.key == VR[i][0])
                v = j;
            if (G.vertices[j].data.key == VR[i][1])
                w = j;
        }
        if (v == -1 || w == -1)
            continue;
        ArcNode *p = (ArcNode *)malloc(sizeof(ArcNode));
        p->adjvex = w;
        p->nextarc = G.vertices[v].firstarc;
        G.vertices[v].firstarc = p;
    }
}
```

```
        G.arcnum++;
    }
    return 1;
}

// DestroyGraph.h
int DestroyGraph(ALGraph &G)
{
    int i;
    for (i = 0; i < G.vexnum; i++)
    {
        ArcNode *p = G.vertices[i].firstarc;
        while (p)
        {
            ArcNode *q = p;
            p = p->nextarc;
            free(q);
        }
        G.vertices[i].firstarc = NULL;
    }
    G.vexnum = 0;
    G.arcnum = 0;
    return OK;
}

// LocateVex.h
int LocateVex(ALGraph G, KeyType u)
{
    int i;
    for (i = 0; i < G.vexnum; i++)
    {
        if (G.vertices[i].data.key == u)
            return i + 1;
    }
    return 0;
}

// PutVex.h
int PutVex(ALGraph &G, KeyType u, VertexType value)
{
    int i;
    for (i = 0; i < G.vexnum; i++)
    {
```

```
        if (G.vertices[i].data.key == u)
        {
            G.vertices[i].data = value;
            return 1;
        }
    }
    return 0;
}

// FirstAdjVex.h
int FirstAdjVex(ALGraph G, KeyType u)
{
    int i;
    for (i = 0; i < G.vexnum; i++)
    {
        if (G.vertices[i].data.key == u)
        {
            if (G.vertices[i].firstarc)
                return G.vertices[i].firstarc->adjvex;
            else
                return -1;
        }
    }
    return -1;
}

// NextAdjVex.h
int NextAdjVex(ALGraph G, KeyType v, KeyType w)
{
    int i, j;
    for (i = 0; i < G.vexnum; i++)
    {
        if (G.vertices[i].data.key == v)
        {
            ArcNode *p = G.vertices[i].firstarc;
            while (p)
            {
                if (G.vertices[p->adjvex].data.key == w)
                {
                    if (p->nextarc)
                        return p->nextarc->adjvex;
                    else

```

```
        return -1;
    }
    p = p->nextarc;
}
}
}
return -1;
}
// InsertVex.h
int InsertVex(ALGraph &G, VertexType v)
{
    if (G.vexnum >= MAX_VERTEX_NUM)
        return 0;
    int i;
    for (i = 0; i < G.vexnum; i++)
    {
        if (G.vertices[i].data.key == v.key)
            return 0;
    }
    G.vertices[G.vexnum].data = v;
    G.vertices[G.vexnum].firstarc = NULL;
    G.vexnum++;
    return 1;
}
// DeleteVex.h
int DeleteVex(ALGraph &G, KeyType v)
{
    int i, j, k = -1;
    for (i = 0; i < G.vexnum; i++)
    {
        if (G.vertices[i].data.key == v)
        {
            k = i;
            break;
        }
    }
    if (k == -1)
        return 0;
    // 删除以v为弧尾的弧
    ArcNode *p = G.vertices[k].firstarc;
```

华中科技大学课程实验报告

```
while (p)
{
    ArcNode *q = p;
    p = p->nextarc;
    free(q);
    G.arcnum--;
}
// 删除以v为弧头的弧
for (i = 0; i < G.vexnum; i++)
{
    if (i == k)
        continue;
    ArcNode *pre = NULL, *cur = G.vertices[i].firstarc;
    while (cur)
    {
        if (cur->adjvex == k)
        {
            if (pre)
                pre->nextarc = cur->nextarc;
            else
                G.vertices[i].firstarc = cur->nextarc;
            ArcNode *tmp = cur;
            cur = cur->nextarc;
            free(tmp);
            G.arcnum--;
        }
        else
        {
            if (cur->adjvex > k)
                cur->adjvex--;
            pre = cur;
            cur = cur->nextarc;
        }
    }
}
// 顶点前移
for (i = k; i < G.vexnum - 1; i++)
{
    G.vertices[i] = G.vertices[i + 1];
}
```



```
G.vexnum--;
return 1;
}
// InsertArc.h
int InsertArc(ALGraph &G, KeyType v, KeyType w)
{
    int i = -1, j = -1, k;
    for (k = 0; k < G.vexnum; k++)
    {
        if (G.vertices[k].data.key == v)
            i = k;
        if (G.vertices[k].data.key == w)
            j = k;
    }
    if (i == -1 || j == -1)
        return 0;
    ArcNode *p = G.vertices[i].firstarc;
    while (p)
    {
        if (p->adjvex == j)
            return 0;
        p = p->nextarc;
    }
    p = (ArcNode *)malloc(sizeof(ArcNode));
    p->adjvex = j;
    p->nextarc = G.vertices[i].firstarc;
    G.vertices[i].firstarc = p;
    G.arcnum++;
    return 1;
}
// DeleteArc.h
int DeleteArc(ALGraph &G, KeyType v, KeyType w)
{
    int i = -1, j = -1, k;
    for (k = 0; k < G.vexnum; k++)
    {
        if (G.vertices[k].data.key == v)
            i = k;
        if (G.vertices[k].data.key == w)
            j = k;
```

```
}
if (i == -1 || j == -1)
    return 0;
ArcNode *pre = NULL, *cur = G.vertices[i].firstarc;
while (cur)
{
    if (cur->adjvex == j)
    {
        if (pre)
            pre->nextarc = cur->nextarc;
        else
            G.vertices[i].firstarc = cur->nextarc;
        free(cur);
        G.arcnum--;
        return 1;
    }
    pre = cur;
    cur = cur->nextarc;
}
return 0;
}
// DFSTraverse.h
void DFS(ALGraph G, int v, int *visited)
{
    visited[v] = 1;
    printf("%d %s ", G.vertices[v].data.key, G.vertices[v].data.others);
    ArcNode *p = G.vertices[v].firstarc;
    while (p)
    {
        if (!visited[p->adjvex])
            DFS(G, p->adjvex, visited);
        p = p->nextarc;
    }
}
int DFSTraverse(ALGraph G)
{
    int visited[MAX_VERTEX_NUM] = {0};
    for (int i = 0; i < G.vexnum; i++)
    {
        if (!visited[i])
```

华中科技大学课程实验报告

```
        DFS(G, i, visited);
    }
    return 1;
}
// BFSTraverse.h
int BFSTraverse(ALGraph G)
{
    int visited[MAX_VERTEX_NUM] = {0};
    int queue[MAX_VERTEX_NUM], front = 0, rear = 0;
    for (int i = 0; i < G.vexnum; i++)
    {
        if (!visited[i])
        {
            visited[i] = 1;
            printf("%d %s ", G.vertices[i].data.key, G.vertices[i].data
.others);
            queue[rear++] = i;
            while (front < rear)
            {
                int v = queue[front++];
                ArcNode *p = G.vertices[v].firstarc;
                while (p)
                {
                    if (!visited[p->adjvex])
                    {
                        visited[p->adjvex] = 1;
                        printf("%d %s ", G.vertices[p->adjvex].data.key
, G.vertices[p->adjvex].data.others);
                        queue[rear++] = p->adjvex;
                    }
                    p = p->nextarc;
                }
            }
        }
    }
    return 1;
}
// VerticesSetLessThanK.h
void dfs_k(ALGraph G, int v, int k, int depth, int *visited)
{

```

```
    if (depth > k)
        return;
    visited[v] = 1;
    printf("%d ", G.vertices[v].data.key);
    ArcNode *p = G.vertices[v].firstarc;
    while (p)
    {
        if (!visited[p->adjvex])
            dfs_k(G, p->adjvex, k, depth + 1, visited);
        p = p->nextarc;
    }
}

int VerticesSetLessThanK(ALGraph G, KeyType v, int k)
{
    int i, start = -1;
    for (i = 0; i < G.vexnum; i++)
    {
        if (G.vertices[i].data.key == v)
        {
            start = i;
            break;
        }
    }
    if (start == -1)
        return 0;
    int visited[MAX_VERTEX_NUM] = {0};
    dfs_k(G, start, k, 0, visited);
    return 1;
}

// AddList.h
int AddList(LISTS &Lists, char ListName[])
{
    if (Lists.length >= 10)
        return 0;
    strcpy(Lists.elem[Lists.length].name, ListName);
    Lists.elem[Lists.length].T.vexnum = 0;
    Lists.elem[Lists.length].T.arcnum = 0;
    Lists.length++;
    return 1;
}
```

```
// LocateList.h
int LocateList(LISTS Lists, char ListName[])
{
    for (int i = 0; i < Lists.length; i++)
    {
        if (strcmp(Lists.elem[i].name, ListName) == 0)
            return i + 1;
    }
    return 0;
}

// RemoveList.h
int RemoveList(LISTS &Lists, char ListName[])
{
    int i, k = -1;
    for (i = 0; i < Lists.length; i++)
    {
        if (strcmp(Lists.elem[i].name, ListName) == 0)
        {
            k = i;
            break;
        }
    }
    if (k == -1)
        return 0;
    for (i = k; i < Lists.length - 1; i++)
    {
        Lists.elem[i] = Lists.elem[i + 1];
    }
    Lists.length--;
    return 1;
}

// ShortestPathLength.h
int ShortestPathLength(ALGraph G, KeyType u, KeyType v)
{
    int start = -1, end = -1;
    for (int i = 0; i < G.vexnum; i++)
    {
        if (G.vertices[i].data.key == u)
            start = i;
        if (G.vertices[i].data.key == v)
```

```
        end = i;
    }
    if (start == -1 || end == -1)
        return -2;
    int dist[MAX_VERTEX_NUM];
    for (int i = 0; i < G.vexnum; i++)
        dist[i] = 1000;
    dist[start] = 0;
    int queue[MAX_VERTEX_NUM], front = 0, rear = 0;
    queue[rear++] = start;
    while (front < rear)
    {
        int cur = queue[front++];
        ArcNode *p = G.vertices[cur].firstarc;
        while (p)
        {
            if (dist[p->adjvex] > dist[cur] + 1)
            {
                dist[p->adjvex] = dist[cur] + 1;
                queue[rear++] = p->adjvex;
            }
            p = p->nextarc;
        }
    }
    if (dist[end] == 1000)
        return 1000;
    return dist[end];
}

// ConnectedComponentsNums.h
void dfs_conn(ALGraph G, int v, int *visited)
{
    visited[v] = 1;
    ArcNode *p = G.vertices[v].firstarc;
    while (p)
    {
        if (!visited[p->adjvex])
            dfs_conn(G, p->adjvex, visited);
        p = p->nextarc;
    }
}
```

华中科技大学课程实验报告

```
int ConnectedComponentsNums(ALGraph G)
{
    int visited[MAX_VERTEX_NUM] = {0};
    int count = 0;
    for (int i = 0; i < G.vexnum; i++)
    {
        if (!visited[i])
        {
            dfs_conn(G, i, visited);
            count++;
        }
    }
    return count;
}

// SaveGraph.h
int SaveGraph(ALGraph G, char FileName[])
{
    FILE *fp = fopen(FileName, "w");
    if (!fp)
        return 0;
    fprintf(fp, "%d %d\n", G.vexnum, G.arcnum);
    for (int i = 0; i < G.vexnum; i++)
    {
        fprintf(fp, "%d %s\n", G.vertices[i].data.key, G.vertices[i].data.others);
    }
    for (int i = 0; i < G.vexnum; i++)
    {
        ArcNode *p = G.vertices[i].firstarc;
        while (p)
        {
            fprintf(fp, "%d %d\n", G.vertices[i].data.key, G.vertices[p->adjvex].data.key);
            p = p->nextarc;
        }
    }
    fclose(fp);
    return OK;
}

// LoadGraph.h
```

华中科技大学课程实验报告

```
int LoadGraph(ALGraph &G, char FileName[])
{
    FILE *fp = fopen(FileName, "r");
    if (!fp)
        return 0;
    int vexnum, arcnum;
    fscanf(fp, "%d %d", &vexnum, &arcnum);
    G.vexnum = vexnum;
    G.arcnum = arcnum;
    for (int i = 0; i < vexnum; i++)
    {
        fscanf(fp, "%d %s", &G.vertices[i].data.key, G.vertices[i].data
.others);
        G.vertices[i].firstarc = NULL;
    }
    for (int i = 0; i < arcnum; i++)
    {
        int v, w;
        fscanf(fp, "%d %d", &v, &w);
        int vi = -1, wi = -1;
        for (int j = 0; j < G.vexnum; j++)
        {
            if (G.vertices[j].data.key == v)
                vi = j;
            if (G.vertices[j].data.key == w)
                wi = j;
        }
        if (vi == -1 || wi == -1)
            continue;
        ArcNode *p = (ArcNode *)malloc(sizeof(ArcNode));
        p->adjvex = vi;
        p->nextarc = G.vertices[vi].firstarc;
        G.vertices[vi].firstarc = p;
    }
    fclose(fp);
    return OK;
}
```